APEX INSTITUTE OF TECHNOLOGY
(AIT)

Department of Computer Science and Engineering

B.E. – Computer Science and Engineering

SEMESTER : 5th

SUBJECT NAME : Advanced Machine Learning Lab

SUBJECT CODE : 24CSH-338

FACULTY : Siddharth Kumar

Contents of Course File

Sr. No.	Particular	Page No.
1	University-Vision and Mission	3
2	Department-Vision and Mission	4
3	PEO	5
4	PO	6
5	PSO	8

6	Course Objectives	8
7	Course Outcomes	9
8	Mapping of COs/POs/PSOs	9 -10
9	Syllabus (As approved in BOS) ---(If Any Changes required, Approval Copy from DAA)	11-12
10	List of Experiments (Mapped with Cos)	12-13
11	**Experiment 1…15 (With Case Studies)** Aim Objective Input/Apparatus Used Procedure/Algorithm/Code Observations/Outcome Discussion Question: Viva Voce	17-119
12	**LAB MST** Question Paper (Mapped with COs) Scheme of Evaluation Sample Answer Sheets CO-Wise Mark Sheets List of Slow Learners and Advance Learners Action Taken	104
13	Best Practices Adopted	
14	Final Award Sheet	
15	List of Slow Learners and Advance Learners	
16	Overall Observation and Action Taken (Recommendation to BOS)	

Vision of Chandigarh University

To be globally recognized as a Centre of Excellence for Research, Innovation, Entrepreneurship and disseminating knowledge by providing inspirational learning to produce professional leaders for serving the society.

Mission of Chandigarh University

Providing world class infrastructure, renowned academicians and ideal environment for Research, Innovation, Consultancy and Entrepreneurship relevant to the society.

Offering programs & courses in consonance with National policies for nation building and meeting global challenges.

Designing Curriculum to match international standards, needs of Industry, civil society and for inculcation of traits of Creative Thinking and Critical Analysis as well as Human and Ethical values.

Ensuring students delight by meeting their aspirations through blended learning, corporate mentoring, professional grooming, flexible curriculum and healthy atmosphere based on co-curricular and extra-curricular activities.

Creating a scientific, transparent and objective examination/evaluation system to ensure an ideal certification.

Establishing strategic relationships with leading National and International corporates and universities for academic as well as research collaborations.

Contributing for creation of healthy, vibrant and sustainable society by involving in Institutional Social Responsibility (ISR) activities like rural development, welfare of senior citizens, women empowerment, community service, health and hygiene awareness and environmental protection

Vision of the Department

To be recognized as a centre of excellence for Computer Science & Engineering education and research, through effective teaching practices, hands-on training on cutting edge computing technologies and excellence in innovation, for creating globally aware competent professionals with strong work ethics whom would be proficient in implementing modern technology solutions and shall have entrepreneurial zeal to solve problems of organizations and society at large.

Mission of the Department

M1: To provide relevant, rigorous and contemporary curriculum and aligned assessment system to ensure effective learning outcomes for engineering technologies.

M2: To provide platform for industry engagement aimed at providing hands-on training on advanced technological and business skills to our students.

M3: To provide opportunities for collaborative, interdisciplinary and cutting-edge research aimed at developing solutions to real life problems

M4: To imbibe quest for innovation, continuous learning and zeal to pursue excellence through hard work and problem-solving approach

M5: To foster skills of leadership, management, communication, team spirit and strong professional ethics in all academic and societal endeavours of our students

Program Education Objectives

PE01 To be able to explore areas of research, technology application & innovation and make a positive impact in different types of institutional settings such as corporate entities, government bodies, NGOs, inter-government organizations, & start-ups.

PEO2 To be able to design, and implement technology and computing solutions to the organizational problems, effectively deploy knowledge of engineering principles, demonstrate critical thinking skills & make the intellectual connections between quantitative and qualitative tools, theories and context to solve the organizational problems

PEO3 To be able to work with, lead & engage big and small teams comprising diverse people in terms of gender, nationality, region, language, culture & beliefs. To understand stated and unstated differences of views, beliefs & customs in diverse & inter disciplinary team settings

PEO4 To be able to continuously learn and update one's knowledge, engage in lifelong learning habits and acquire latest knowledge to perform in current work settings

PEO5 To continuously strive for justice, ethics, equality, honesty, and integrity both in personal and professional pursuits. Able to understand and conduct in a way that is responsible and respectful.

Program Outcomes

PO1 Engineering Knowledge: Apply knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.

PO2 Problem Analysis: Identify, formulate, research literature and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences.

PO3 Design/ Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal and environmental considerations.

PO4 Conduct investigations of complex problems using research-based knowledge and research methods including design of experiments, analysis and interpretation of data and synthesis of information to provide valid conclusions.

PO5 Modern Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6 The Engineer and Society: Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to professional engineering practice.

PO7 Environment and Sustainability: Understand the impact of professional engineering solutions in societal and environmental contexts and demonstrate knowledge of and need for sustainable development.

PO8 Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.

PO9 Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams and in multi-disciplinary settings.

PO10 Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations and give and receive clear instructions.

PO11 Project Management and Finance: Demonstrate knowledge and understanding of engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12 Life-long Learning: Recognize the need for and have the preparation and ability to Engage in independent and life- long learning in the broadest context of technological Change.

PO13 Demonstrate the capability to apply analytic thought to a body of knowledge, including the analysis and evaluation of policies, and practices. Identify relevant assumptions or implications, logical flaws and loopholes in the presented arguments.

PO14 Demonstrate to create, perform, or think in different and diverse ways about the given scenario. Innovate and perform tasks in a better manner, view a problem or a situation from multiple perspectives, think ‘out of the box’ and generate solutions to complex problems in unfamiliar contexts.

PO15 Demonstrate the ability to identify with or understand the perspective, experiences, or points of view of another individual or group, and to identify and understand other people’s emotions.

PO16 Demonstrate the ability to participate in community-engaged services/ activities for promoting the well-being of society.

PO17 Demonstrate the acquisition of knowledge of the values and beliefs of multiple cultures, capability to effectively engage in a multicultural group/society and interact respectfully with diverse groups and gender sensitivity and adopting a gender-neutral approach, as also empathy for the less advantaged and the differently-abled including those with learning disabilities.

Program Specific (Learning) Outcome for Information Security

Specifically, the Program Specific (Learning) Outcomes (PSOs) of the Information Security program are that upon graduation students will be able to:

PSO1 Analyse and evaluate systems with respect to maintaining operations in the presence of risks and threats, and also communicate the human role in security systems with an emphasis on ethics, social engineering vulnerabilities and training.

PSO2 Conduct an Information Security risk assessment an Audit and troubleshoot Information Security systems using Cryptographic measures

PSO3 Be able to assess the Ethical Ramification of working in Information Security, information assurance, and cyber/computer forensics software/tools.

PSO4 Design and develop a security architecture and operational and strategic cyber security strategies and policies.

COURSE OBJECTIVES

The Course aims to:

1. To develop a strong understanding of advanced ensemble models, including voting, bagging, boosting, and blending techniques for predictive modelling.
2. To provide an in-depth understanding of supervised and unsupervised learning architectures, including neural networks and clustering methods such as AGNES and DIANA.
3. To equip learners with knowledge of probabilistic graphical models, including factor graphs, belief networks, and sampling-based reasoning methods.
4. To cultivate the ability to implement distributed and federated learning architectures, frameworks, and real-world machine learning applications.
5. To expose students to explainable AI and image generation techniques, including SHAP, LIME, attention mechanisms, image segmentation, GANs, and object detection.

COURSE OUTCOMES

On completion of this course, the students shall be able to

1. Recall the foundational concepts, terminology, and mathematical principles of advanced supervised algorithms, unsupervised learning methods, ensemble learning, federated learning, image generation, and explainable AI.
2. Illustrate the working mechanisms and characteristics of Ridge, Lasso, Group Lasso, SVM variants, SVR, One-Class SVM, LS-SVM, hierarchical clustering, SOM, Gaussian Mixture Models, ensemble methods, probabilistic models, and XAI techniques.
3. Apply appropriate supervised, unsupervised, ensemble, probabilistic, distributed, federated, image generation, and explainable AI techniques to solve real-world machine learning problems.
4. Analyse and evaluate machine learning models using suitable performance parameters and cross-validation to identify the best-suited techniques for specific applications.
5. Design and implement optimized machine learning solutions by integrating advanced supervised algorithms, unsupervised learning methods, ensemble learning, federated learning, image generation, and explainable AI for complex predictive and decision-making problems.

Mapping of COs/POs/PSOs

Mapping Between COs and Pos		
SN	**Course Outcome (CO)**	**Mapped Program Outcome (PO)**
1	Recall the foundational concepts, terminology, and mathematical principles of advanced supervised algorithms, unsupervised learning methods, ensemble learning, federated learning, image generation, and explainable AI.	1 to 6, 9 to 12
2	Illustrate the working mechanisms and characteristics of Ridge, Lasso, Group Lasso, SVM variants, SVR, One-Class SVM, LS-SVM, hierarchical clustering, SOM, Gaussian Mixture Models, ensemble methods, probabilistic models, and XAI techniques.	1 to 6, 9 to 12
3	Apply appropriate supervised, unsupervised, ensemble, probabilistic, distributed, federated, image generation, and explainable AI techniques to solve real-world machine learning problems.	1 to 6, 9 to 12
4.	Analyse and evaluate machine learning models using suitable performance parameters and cross-validation to identify the best-suited techniques for specific applications.	1 to 6, 9 to 12

5.	Design and implement optimized machine learning solutions by integrating advanced supervised algorithms, unsupervised learning methods, ensemble learning, federated learning, image generation, and explainable AI for complex predictive and decision-making problems.	1 to 6, 9 to 12

1=addressed to small extent
2= addressed significantly
3=major part of course

Syllabus

SN	**Course Code:**	**Course Name:** Advanced Machine Learning	**L**	**T**	**P**	**S**	**C**	**CH**	**Course Category**	**Course Type**
1	24CSH-338	**Course Coordinator**: Mr. Siddharth Kumar	3	0	2	0	4	5	PC*	Hybrid
Pre-Requisites:		Introduction to Python Programming Skills and Machine Learning								
Co-Requisite		Big Data Technologies								
Anti-Requisite										

COURSE DESCRIPTION

This course delves into advanced supervised and unsupervised learning models, focusing on ensemble learning techniques like bagging (Random Forest), boosting (Adaboost), and blending. It also covers foundational algorithms like neural networks and advanced clustering methods such as AGNES, DIANA, and Gaussian Mixture Models. Further, the course explores probabilistic graphical models, distributed and federated learning, and their real-world applications. In the domain of image generation, students will study image segmentation, captioning, and generative adversarial networks (GANs). Additionally, the course emphasizes explainable AI (XAI), covering techniques like SHAP, LIME, and attention mechanisms to promote transparency and trust in AI models.

TEXT BOOKS

T1: Massaron, L., & Boschetti, A. (2016). Regression analysis with Python. Packt Publishing Ltd.

T2: Jajuga, K., Sokolowski, A., & Bock, H. H. (Eds.). (2012). Classification, clustering, and data analysis: recent advances and applications.

T3: Bishop, C. M., & Bishop, H. (2023). Deep learning: Foundations and concepts. Springer Nature.

T4: Andreas C. Müller, Sarah Guido, — "Introduction to Machine Learning with Python",O'Reilly Media, Inc.; 2016.

REFERENCE BOOKS

R1: Matloff, N. (2017). Statistical regression and classification: from linear models to machine learning. Chapman and Hall/CRC.

R2: Jajuga, K., Sokolowski, A., & Bock, H. H. (Eds.). (2012). Classification, clustering, and data analysis: recent advances and applications.

R3: Vishnoi, V. K., Chauhan, N. R., & Kumar, K. (2024). An introduction to deep learning. Xoffencerpublication.

R4: Paul Gries, Jennifer Campbell and Jason Montojo, ―Practical Programming: An Introduction to Computer Science using Python 3, Second edition, Pragmatic Programmers, LLC, 2013.

COURSE CONTENT

Unit-1	**Advanced Supervised and Unsupervised Models**	**Contact Hours: 15**
Advanced Supervised Algorithms	Ridge, Lasso, and Group Lasso, Variants of SVM, Support Vector Regression (SVR), One-Class SVM, Least Squares SVM (LS-SVM), SVM in high-dimensional feature spaces.	
Unsupervised Learning Algorithms	Introduction to clustering: Need and Role of clustering, Hierarchical Clustering: AGNES, DIANA, Topology-based Clustering: Self-Organizing Map, Probabilistic Sequence/Latent Methods: Gaussian Mixture Models	
Advanced Ensemble Learning Algorithms	Ensemble Learning Basics: Combination Schemes, Voting, Weighted Averaging, Voting Strategies: Majority Voting, Hard and Soft Voting, Cross-validation, Types and Significance, Bagging Method: Random Forest, Boosting Method: Adaboost, Gradient Boosting, XGBoost, Stacking and blending methods	
Self-Study Topic	-	
Unit-2	**Probabilistic, Distributed and Federated Learning Architectures**	**Contact Hours: 15**
Probabilistic Graphical Models	Introduction to Probabilistic Graphical Model: Factor Graphs and Belief Networks, Deep Probabilistic Models: Deep belief networks and Boltzmann machines, Sampling in Probabilistic Learning: Gibbs Sampling.	
Distributed and Federated Learning	Distributed Machine Learning (DML) and its Types, Distributed Training Architecture, Frameworks for DML, Challenges in DML, Federated Learning (FL), Need of Federated Learning, Architecture, Challenges and real-world applications.	
Self-Study Topic	-	
Unit-3	**Image Generation and Explainable AI**	**Contact Hours: 15**
Image Generation	Computer Vision Foundation: Image segmentation, Automatic Image Captioning, Image generation with Generative adversarial networks, Object detection, Attention models for computer vision tasks.	
Explainable AI	Explainable AI (XAI) Techniques, Interpretable ML Models: SHAP, LIME, Attention Mechanisms, Regulatory Compliance & Trustworthy AI, Privacy and Security in ML.	

Self-Study Topic	-

MODE OF EVALUATION: The performance of students is evaluated as follows:

	Theory	
Components	**Continuous Internal Assessment (CAE)**	**Semester End Examination (SEE)**
Marks	60	40
Total Marks	100	

CO vs PO/PSO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PO13	PO14	PO15	PO16	PO17	PSO1	PSO2	PSO3
CO1	2	2	2	2	3	NA	NA	NA	NA	NA	1	1	NA	NA	NA	NA	NA	3	3	3
CO2	2	2	2	2	2	NA	NA	NA	NA	NA	1	1	NA	NA	NA	NA	NA	2	2	2
CO3	1	1	1	1	1	NA	NA	NA	NA	NA	1	1	NA	NA	NA	NA	NA	3	3	3
CO4	1	1	1	2	2	NA	NA	NA	NA	NA	1	1	NA	NA	NA	NA	NA	3	3	3
CO5	2	2	2	1	1	NA	NA	NA	NA	NA	1	1	NA	NA	NA	NA	NA	3	3	3
Target	1.6	1.6	1.6	1.6	1.8	NA	NA	NA	NA	NA	1	1	NA	NA	NA	NA	NA	2.8	2.8	2.8

List of Experiments (Mapped with Cos)

Unit No	Exp No	Topic/Sub-Topic	Text/ Reference Books	Pedagogical Tool	Mapped with CO Numer (s)	BT Level
1	**1**	To implement Ridge Regression and Lasso Regression using Python and compare their performance with Linear Regression on a regression dataset.	T1, R1	Simulation, PPT and WhiteBoard	CO3	BT3, BT4
1	**2**	To implement a Support Vector Machine (SVM) classifier on a	T1, R1	Simulation, PPT and WhiteBoard	CO3	BT3, BT4

		classification dataset and study the effect of different kernels on the classification accuracy using Python				
1	**3**	Implement AGNES (Agglomerative Nesting) and DIANA (Divisive Analysis) for clustering a dataset.	T1, R1	Simulation, PPT and WhiteBoard	CO3	BT3, BT4
1	**4**	**Case Study:** To implement a Self-Organizing Map (SOM) using Python and apply it for clustering and visualization of high-dimensional data.	T1, R1	Simulation, PPT and WhiteBoard	CO3	BT3, BT4
1	**5**	**Case Study:** Implement ensemble learning techniques on a real-world dataset and evaluate their performance using cross-validation and analyse the impact of different hyperparameters on model accuracy.	T1, R1	Simulation, PPT and WhiteBoard	CO3	BT3, BT4
2	**6**	Implement a Deep Belief Network on a simple dataset	T2, R2	Simulation, PPT and WhiteBoard	CO4	BT4, BT5

		and visualize the network learned features.				
2	**7**	To implement Gibbs Sampling in Python and understand how it is used to generate samples from a probability distribution when direct sampling is difficult.	T2, R2	Simulation, PPT and WhiteBoard	CO4	BT4, BT5
2	**8**	**Case Study:** Implement a Hidden Markov Model (HMM) to predict sequences in time-series data (e.g., weather forecasting, speech recognition).	T2, R2	Simulation, PPT and WhiteBoard	CO4	BT4, BT5
2	**9**	**Case Study:** Implement a Federated Learning system using a machine learning model.	T2, R2	Simulation, PPT and WhiteBoard	CO4	BT4, BT5
2	**10**	To Implement a Federated Learning experiment by dividing one dataset into multiple clients, training models separately, and combining	T2, R2	Simulation, PPT and WhiteBoard	CO4	BT4, BT5

		their predictions.				
3	**11**	To implement a simple image segmentation experiment using Python and OpenCV to separate an object or region of interest from an image.	T3, R3	Simulation, PPT and WhiteBoard	CO5	BT4, BT5
3	**12**	Implement a basic Generative Adversarial Network (GAN) to generate synthetic images from random noise.	T3, R3	Simulation, PPT and WhiteBoard	CO5	BT4, BT5
3	**13**	Implement SHAP and LIME techniques to explain the predictions of a Random Forest Model on a classification task.	T3, R3	Simulation, PPT and WhiteBoard	CO5	BT4, BT5
3	**14**	Apply an object detection model to detect and classify objects in an image dataset.	T3, R3	Simulation, PPT and WhiteBoard	CO5	BT4, BT5
3	**15**	**Case Study:** Vehicle Detection in Road Traffic using the concept of Object Detection.	T3, R3	Simulation, PPT and WhiteBoard	CO5	BT4, BT5

Practical 1

Mapped Course Outcomes - CO3

CO3: Apply appropriate supervised, unsupervised, ensemble, probabilistic, distributed, federated, image generation, and explainable AI techniques to solve real-world machine learning problems.

Manual of the experiment.

Aim

To implement **Ridge Regression** and **Lasso Regression** using Python and compare their performance with **Linear Regression** on a regression dataset.

Objective

- To understand the concept of regularization in regression.
- To implement Linear Regression, Ridge Regression, and Lasso Regression.
- To compare the models using performance metrics.
- To analyze the effect of **L1** and **L2** regularization.
- To identify which model gives better performance on the selected dataset.

Tools Required

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Scikit-learn

Theory

Linear Regression is a supervised learning algorithm used to predict continuous values. It tries to find the best-fit line by minimizing the difference between actual and predicted values.

However, Linear Regression may overfit when the dataset has many features or correlated features. To solve this, regularization techniques are used.

Ridge Regression adds an **L2 penalty** to the loss function. It reduces the magnitude of coefficients but usually does not make them exactly zero. Scikit-learn describes Ridge as linear least squares with L2 regularization.

Lasso Regression adds an **L1 penalty** to the loss function. It can shrink some coefficients exactly to zero, so it also helps in feature selection. Scikit-learn notes that Lasso estimates sparse coefficients using L1 regularization.

Methodology

1. Import required Python libraries.
2. Load a regression dataset.
3. Split the dataset into training and testing data.
4. Apply feature scaling.
5. Train three models:
    - Linear Regression
    - Ridge Regression
    - Lasso Regression
6. Predict output values using test data.
7. Evaluate models using:
    - Mean Squared Error
    - Root Mean Squared Error
    - R^2 Score
8. Compare the performance of all models.
9. Display results in tabular and graphical form.

Dataset

Dataset used: **Diabetes Dataset from Scikit-learn**

The diabetes dataset is a regression dataset with **442 samples** and **10 numeric features**. The target value is a quantitative measure of disease progression one year after baseline.

Features include:

- Age
- Sex
- Body Mass Index
- Average Blood Pressure
- Serum measurements
- Blood sugar level

Code

```
# Experiment: Linear Regression vs Ridge Regression vs Lasso Regression

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

# 1. Load dataset
diabetes = load_diabetes()
X = diabetes.data
y = diabetes.target

# 2. Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 4. Create models
linear_model = LinearRegression()
ridge_model = Ridge(alpha=1.0)
lasso_model = Lasso(alpha=0.1, max_iter=10000)

# 5. Train models
linear_model.fit(X_train_scaled, y_train)
ridge_model.fit(X_train_scaled, y_train)
lasso_model.fit(X_train_scaled, y_train)

# 6. Make predictions
linear_pred = linear_model.predict(X_test_scaled)
ridge_pred = ridge_model.predict(X_test_scaled)
lasso_pred = lasso_model.predict(X_test_scaled)

# 7. Evaluation function
def evaluate_model(name, y_test, y_pred):
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
```

```
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    return {
        "Model": name,
        "MAE": mae,
        "MSE": mse,
        "RMSE": rmse,
        "R2 Score": r2
    }

# 8. Store results
results = [
    evaluate_model("Linear Regression", y_test, linear_pred),
    evaluate_model("Ridge Regression", y_test, ridge_pred),
    evaluate_model("Lasso Regression", y_test, lasso_pred)
]

results_df = pd.DataFrame(results)

print("Model Performance Comparison:")
print(results_df)

# 9. Compare coefficients
coef_df = pd.DataFrame({
    "Feature": diabetes.feature_names,
    "Linear Regression": linear_model.coef_,
    "Ridge Regression": ridge_model.coef_,
    "Lasso Regression": lasso_model.coef_
})

print("\nCoefficient Comparison:")
print(coef_df)

# 10. Plot actual vs predicted values
plt.figure(figsize=(8, 5))
plt.scatter(y_test, linear_pred, label="Linear Regression")
plt.scatter(y_test, ridge_pred, label="Ridge Regression")
plt.scatter(y_test, lasso_pred, label="Lasso Regression")
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Actual vs Predicted Values")
plt.legend()
plt.show()

# 11. Bar graph for R2 score
plt.figure(figsize=(7, 4))
plt.bar(results_df["Model"], results_df["R2 Score"])
plt.xlabel("Model")
```

```
plt.ylabel("R2 Score")
plt.title("R2 Score Comparison")
plt.xticks(rotation=20)
plt.show()
```

Results

Sample expected result may look like this:

Model	MAE	MSE	RMSE	R^2 Score
Linear Regression	42.79	2900.19	53.85	0.45
Ridge Regression	42.73	2892.01	53.78	0.45
Lasso Regression	42.78	2898.63	53.83	0.45

The exact values may change slightly depending on the random state, alpha value, and dataset split.

Outcome of the Experiment

- Linear Regression, Ridge Regression, and Lasso Regression were successfully implemented.
- Ridge Regression reduced the effect of large coefficients using L2 regularization.
- Lasso Regression reduced some coefficients using L1 regularization and can help in feature selection.
- Ridge and Lasso help control overfitting compared to simple Linear Regression.
- The performance of all three models was compared using MAE, MSE, RMSE, and R^2 Score.
- Ridge Regression generally performs better when features are correlated, while Lasso is useful when feature selection is required.

Relevant Demonstrative Video Link

- Ridge and Lasso Regression using Python and Scikit-learn:
- Ridge and Lasso Regression Intuition:
- L1 and L2 Regularization Tutorial:

Other Resource Materials / Web Links

- Scikit-learn Ridge Regression Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html

- Scikit-learn Linear Models User Guide: https://scikit-learn.org/stable/modules/linear_model.html
- Scikit-learn Diabetes Dataset Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_diabetes.html
- Scikit-learn Ordinary Least Squares and Ridge Regression Example: https://scikit-learn.org/stable/auto_examples/linear_model/plot_ols_ridge.html
- DataCamp Ridge and Lasso Regression Tutorial: https://www.datacamp.com/tutorial/tutorial-lasso-ridge-regression

Questions to be asked for Viva Voce:

Q1. Define Linear Regression and mention its use in prediction problems.

Q2. Explain the need of regularization in regression models.

Q3. Differentiate Ridge Regression and Lasso Regression.

Q4. Describe the role of the regularization parameter alpha in Ridge and Lasso Regression.

Q5. Explain the effect of Ridge Regression on model coefficients.

Q6. Explain the effect of Lasso Regression on feature selection.

Q7. Compare the performance of Linear Regression, Ridge Regression and Lasso Regression.

Q8. Identify suitable evaluation metrics for regression models.

Q9. Analyse the impact of overfitting in Linear Regression.

Q10. Discuss the advantages of using Ridge and Lasso Regression on real-world datasets.

Practical 2

Mapped Course Outcomes - CO3

CO3: Apply appropriate supervised, unsupervised, ensemble, probabilistic, distributed, federated, image generation, and explainable AI techniques to solve real-world machine learning problems.

Manual of the experiment.

Aim

To implement a **Support Vector Machine (SVM) classifier** on a classification dataset and study the effect of different kernels on the classification accuracy using Python.

Objective

- To understand the working of Support Vector Machine classification.
- To implement SVM using Python.
- To apply different SVM kernels such as **linear**, **polynomial**, **RBF**, and **sigmoid**.
- To compare classification accuracy for different kernels.
- To analyze how kernel choice affects model performance.

Tools Required

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Scikit-learn

Theory

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression. In classification, SVM finds the best decision boundary, called a **hyperplane**, that separates data points of different classes.

The data points closest to the hyperplane are called **support vectors**. These points are important because they decide the position and orientation of the hyperplane.

SVM can classify both linear and non-linear data using **kernel functions**. A kernel transforms data into a higher-dimensional space where it becomes easier to separate classes. Scikit-learn's SVC supports kernels such as linear, poly, rbf, and sigmoid; if no kernel is specified, rbf is used by default.

Common SVM kernels are:

Kernel	Description
Linear Kernel	Used when data is linearly separable
Polynomial Kernel	Used when data has curved decision boundaries
RBF Kernel	Used for complex non-linear classification
Sigmoid Kernel	Similar to neural network activation behavior

SVM is effective in high-dimensional spaces and uses support vectors in the decision function, making it memory efficient.

Methodology

1. Import the required Python libraries.
2. Load a classification dataset.
3. Split the dataset into training and testing sets.
4. Apply feature scaling.
5. Train SVM classifiers using different kernels:
    - Linear
    - Polynomial
    - RBF
    - Sigmoid
6. Predict class labels for test data.
7. Calculate accuracy and classification report.
8. Compare the performance of different kernels.
9. Display the results in tabular and graphical form.

Dataset

Dataset used: **Iris Dataset**

The Iris dataset is a classic multi-class classification dataset available in Scikit-learn. It contains **150 samples**, **3 classes**, and **4 numeric features**. Each class has 50 samples.

The features are:

- Sepal length
- Sepal width
- Petal length

- Petal width

The target classes are:

- Setosa
- Versicolor
- Virginica

Code

```
# Experiment: SVM Classifier with Different Kernels

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# 1. Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# 2. Convert to DataFrame for better understanding
df = pd.DataFrame(X, columns=iris.feature_names)
df["target"] = y

print("First five rows of dataset:")
print(df.head())

print("\nTarget Classes:")
print(iris.target_names)

# 3. Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 4. Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 5. Define kernels
```

```
kernels = ["linear", "poly", "rbf", "sigmoid"]

results = []

# 6. Train and evaluate SVM for each kernel
for kernel in kernels:
    model = SVC(kernel=kernel, random_state=42)
    model.fit(X_train_scaled, y_train)

    y_pred = model.predict(X_test_scaled)

    accuracy = accuracy_score(y_test, y_pred)

    results.append({
        "Kernel": kernel,
        "Accuracy": accuracy
    })

    print("\n-----------------------------------")
    print("Kernel:", kernel)
    print("Accuracy:", accuracy)
    print("Confusion Matrix:")
    print(confusion_matrix(y_test, y_pred))
    print("Classification Report:")
    print(classification_report(y_test, y_pred, target_names=iris.target_names))

# 7. Convert results to DataFrame
results_df = pd.DataFrame(results)

print("\nAccuracy Comparison:")
print(results_df)

# 8. Plot accuracy comparison
plt.figure(figsize=(7, 4))
plt.bar(results_df["Kernel"], results_df["Accuracy"])
plt.xlabel("SVM Kernel")
plt.ylabel("Accuracy")
plt.title("Accuracy Comparison of SVM Kernels")
plt.ylim(0, 1.1)
plt.show()
```

Results

Sample expected output:

Kernel	**Accuracy**
Linear	0.96

Polynomial	0.93
RBF	0.96
Sigmoid	0.90

The exact accuracy may vary slightly depending on train-test split, scaling, and parameter values.

Outcome of the Experiment

- SVM classifier was successfully implemented using Python.
- Different kernels were applied and compared.
- The **linear kernel** performed well when the data was nearly linearly separable.
- The **RBF kernel** performed well for non-linear boundaries.
- The **polynomial kernel** captured curved relationships in the data.
- The **sigmoid kernel** was less stable compared to linear and RBF kernels in many cases.
- Kernel selection has a direct effect on classification accuracy.
- SVM performance improves when features are properly scaled.

Relevant Demonstrative Video Link

- SVM with Python and Scikit-learn:
- SVM using Iris Dataset in Python:
- Support Vector Machine in Python:

Other Resource Materials / Web Links

- Scikit-learn SVC Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html
- Scikit-learn Support Vector Machines User Guide: https://scikit-learn.org/stable/modules/svm.html
- Scikit-learn Iris Dataset Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_iris.html
- Scikit-learn Example: Classification Boundaries with Different SVM Kernels: https://scikit-learn.org/stable/auto_examples/svm/plot_svm_kernels.html
- DataCamp SVM Classification Tutorial: https://www.datacamp.com/tutorial/svm-classification-scikit-learn-python

Questions to be asked for Viva Voce:

Q1. Define Support Vector Machine and its role in classification.

Q2. Explain the concept of hyperplane in SVM.

Q3. Describe the role of support vectors in SVM classification.

Q4. Differentiate linear kernel, polynomial kernel and RBF kernel.

Q5. Explain the effect of kernel selection on classification accuracy.

Q6. Identify the importance of margin maximization in SVM.

Q7. Analyze the role of C parameter in SVM.

Q8. Discuss the effect of gamma in RBF kernel.

Q9. Compare SVM performance with different kernels on a classification dataset.

Q10. Evaluate the advantages and limitations of SVM classifier.

Practical 3

Mapped Course Outcomes - CO3

CO3: Apply appropriate supervised, unsupervised, ensemble, probabilistic, distributed, federated, image generation, and explainable AI techniques to solve real-world machine learning problems.

Manual of the experiment.

Aim

To implement **AGNES (Agglomerative Nesting)** and **DIANA (Divisive Analysis)** for clustering a dataset using Python.

Objective

- To understand hierarchical clustering.
- To implement **AGNES**, a bottom-up hierarchical clustering method.
- To implement **DIANA**, a top-down hierarchical clustering method.
- To visualize clusters using dendrograms.
- To compare the working of agglomerative and divisive clustering.
- To evaluate clustering performance using suitable metrics.

Tools Required

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Scikit-learn
- SciPy

Theory

Hierarchical clustering is an unsupervised machine learning technique used to group similar data points into clusters. It creates a tree-like structure called a **dendrogram**, which shows how clusters are formed or divided.

There are two main types of hierarchical clustering:

1. AGNES: Agglomerative Nesting

AGNES is a bottom-up clustering approach.

It starts by treating each data point as a separate cluster. Then, it repeatedly merges the two closest clusters until all data points form one large cluster or the desired number of clusters is reached.

Steps of AGNES:

- Start with each data point as an individual cluster.
- Calculate distance between clusters.
- Merge the two nearest clusters.
- Repeat until required clusters are formed.

Common linkage methods:

- Single linkage
- Complete linkage
- Average linkage
- Ward linkage

2. DIANA: Divisive Analysis

DIANA is a top-down clustering approach.

It starts with all data points in one single cluster. Then, it repeatedly splits clusters into smaller clusters until each data point becomes a separate cluster or the desired number of clusters is reached.

Steps of DIANA:

- Start with all data points in one cluster.
- Find the most dissimilar data point.
- Split it into a new cluster.
- Move other dissimilar points to the new cluster.
- Repeat until required clusters are formed.

Methodology

1. Import required Python libraries.
2. Load a clustering dataset.
3. Select relevant numerical features.
4. Apply feature scaling.
5. Implement AGNES using AgglomerativeClustering.
6. Plot dendrogram for AGNES using SciPy.
7. Implement a simple DIANA clustering function.

8. Assign cluster labels for both methods.
9. Visualize clusters using scatter plots.
10. Compare AGNES and DIANA results.

Dataset

Dataset used: **Iris Dataset**

The Iris dataset contains measurements of iris flowers. It has 150 samples and 4 numerical features:

- Sepal length
- Sepal width
- Petal length
- Petal width

For clustering, labels are not used during training because clustering is an unsupervised learning method.

Code

```
# Experiment: AGNES and DIANA Clustering

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import dendrogram, linkage
from scipy.spatial.distance import pdist, squareform

# 1. Load dataset
iris = load_iris()
X = iris.data

df = pd.DataFrame(X, columns=iris.feature_names)

print("First five rows of dataset:")
print(df.head())

# 2. Feature scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
# ------------------------------------------------------
# AGNES: Agglomerative Nesting
# ------------------------------------------------------

# 3. Dendrogram for AGNES
linked = linkage(X_scaled, method='ward')

plt.figure(figsize=(10, 5))
dendrogram(linked)
plt.title("Dendrogram using AGNES")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()

# 4. Apply AGNES clustering
agnes = AgglomerativeClustering(n_clusters=3, linkage='ward')
agnes_labels = agnes.fit_predict(X_scaled)

df["AGNES Cluster"] = agnes_labels

# 5. Evaluate AGNES
agnes_score = silhouette_score(X_scaled, agnes_labels)

print("\nAGNES Cluster Labels:")
print(agnes_labels)

print("\nAGNES Silhouette Score:", agnes_score)

# ------------------------------------------------------
# DIANA: Divisive Analysis - Simple Implementation
# ------------------------------------------------------

def diana_clustering(X, n_clusters=3):
    """
    Simple DIANA clustering implementation.
    Starts with one cluster and repeatedly splits the cluster
    having the largest diameter.
    """

    clusters = [list(range(len(X)))]

    while len(clusters) < n_clusters:
        # Find cluster with largest diameter
        max_diameter = -1
        cluster_to_split_index = None

        for i, cluster in enumerate(clusters):
            if len(cluster) <= 1:
```

```
        continue

    distances = squareform(pdist(X[cluster]))
    diameter = np.max(distances)

    if diameter > max_diameter:
        max_diameter = diameter
        cluster_to_split_index = i

if cluster_to_split_index is None:
    break

cluster_to_split = clusters.pop(cluster_to_split_index)

# Distance matrix of selected cluster
sub_X = X[cluster_to_split]
distances = squareform(pdist(sub_X))

# Find most dissimilar point
avg_distances = np.mean(distances, axis=1)
splinter_index = np.argmax(avg_distances)

splinter_group = [cluster_to_split[splinter_index]]
remaining_group = [
    point for idx, point in enumerate(cluster_to_split)
    if idx != splinter_index
]

changed = True

while changed:
    changed = False

    for point in remaining_group[:]:
        point_position = cluster_to_split.index(point)

        dist_to_splinter = np.mean([
            np.linalg.norm(X[point] - X[s])
            for s in splinter_group
        ])

        dist_to_remaining = np.mean([
            np.linalg.norm(X[point] - X[r])
            for r in remaining_group
            if r != point
        ]) if len(remaining_group) > 1 else 0

        if dist_to_splinter < dist_to_remaining:
            splinter_group.append(point)
```

```
                remaining_group.remove(point)
                changed = True

        clusters.append(remaining_group)
        clusters.append(splinter_group)

    labels = np.zeros(len(X), dtype=int)

    for cluster_id, cluster in enumerate(clusters):
        for point in cluster:
            labels[point] = cluster_id

    return labels

# 6. Apply DIANA clustering
diana_labels = diana_clustering(X_scaled, n_clusters=3)

df["DIANA Cluster"] = diana_labels

# 7. Evaluate DIANA
diana_score = silhouette_score(X_scaled, diana_labels)

print("\nDIANA Cluster Labels:")
print(diana_labels)

print("\nDIANA Silhouette Score:", diana_score)

# ------------------------------------------------------
# Visualization
# ------------------------------------------------------

# 8. Visualize AGNES clusters using first two features
plt.figure(figsize=(7, 5))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=agnes_labels)
plt.title("AGNES Clustering")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()

# 9. Visualize DIANA clusters using first two features
plt.figure(figsize=(7, 5))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=diana_labels)
plt.title("DIANA Clustering")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()

# 10. Comparison table
comparison = pd.DataFrame({
```

```
    "Method": ["AGNES", "DIANA"],
    "Number of Clusters": [len(np.unique(agnes_labels)), len(np.unique(diana_labels))],
    "Silhouette Score": [agnes_score, diana_score]
})

print("\nComparison of AGNES and DIANA:")
print(comparison)
```

Results

Sample output:

Method	Number of Clusters	Silhouette Score
AGNES	3	0.44
DIANA	3	0.35

The exact values may change depending on the dataset, scaling method, and clustering parameters.

The dendrogram shows how clusters are merged in AGNES. The scatter plots show how data points are grouped by AGNES and DIANA.

Outcome of the Experiment

- AGNES and DIANA clustering methods were successfully implemented.
- AGNES followed a bottom-up approach by merging smaller clusters.
- DIANA followed a top-down approach by splitting larger clusters.
- A dendrogram was generated for hierarchical clustering.
- Clusters were visualized using scatter plots.
- Silhouette score was used to compare clustering quality.
- AGNES generally gives stable results using standard libraries, while DIANA requires custom implementation because it is less commonly available in Python libraries.

Relevant Demonstrative Video Link

- Hierarchical Clustering Explained:
- Agglomerative Hierarchical Clustering:
- Hierarchical Clustering with Python:

Other Resource Materials / Web Links

- Scikit-learn Agglomerative Clustering Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.cluster.AgglomerativeClustering.html
- Scikit-learn Clustering User Guide: https://scikit-learn.org/stable/modules/clustering.html
- SciPy Hierarchical Clustering Documentation: https://docs.scipy.org/doc/scipy/reference/cluster.hierarchy.html
- Scikit-learn Iris Dataset Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_iris.html
- Hierarchical Clustering Tutorial: https://www.geeksforgeeks.org/machine-learning/hierarchical-clustering/

Questions to be asked for Viva Voce:

Q1. Define hierarchical clustering and mention its major types.

Q2. Explain the working of AGNES clustering.

Q3. Explain the working of DIANA clustering.

Q4. Differentiate agglomerative and divisive clustering.

Q5. Describe the role of distance measures in hierarchical clustering.

Q6. Explain the use of dendrogram in clustering analysis.

Q7. Compare single linkage, complete linkage and average linkage methods.

Q8. Analyse the effect of number of clusters on clustering results.

Q9. Discuss suitable datasets for AGNES and DIANA implementation.

Q10. Evaluate the advantages and limitations of hierarchical clustering.

Practical 4

Mapped Course Outcomes - CO3

CO3: Apply appropriate supervised, unsupervised, ensemble, probabilistic, distributed, federated, image generation, and explainable AI techniques to solve real-world machine learning problems.

Manual of the experiment.

Aim

To implement a **Self-Organizing Map (SOM)** using Python and apply it for clustering and visualization of high-dimensional data.

Objective

- To understand the concept of Self-Organizing Map.
- To implement SOM using Python.
- To apply SOM on high-dimensional data.
- To cluster similar data points using SOM.
- To visualize high-dimensional data on a two-dimensional map.
- To analyze the role of Best Matching Unit and neighborhood learning.

Tools Required

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Scikit-learn
- MiniSom library

Theory

A **Self-Organizing Map (SOM)** is an unsupervised neural network technique used for clustering and visualization of high-dimensional data. It is also known as a **Kohonen Map**.

SOM maps high-dimensional input data onto a low-dimensional grid, usually a two-dimensional grid, while preserving the topological relationship between data points. Similar data points are placed close to each other on the map.

SOM uses competitive learning. For each input vector, the neuron whose weight vector is closest to the input is selected. This neuron is called the **Best Matching Unit (BMU)**. The BMU and its neighboring neurons are updated to become more similar to the input vector.

SOM is useful for clustering, visualization, dimensionality reduction, anomaly detection, and pattern recognition. MiniSom is a minimal NumPy-based Python implementation of Self-Organizing Maps.

Methodology

1. Import required Python libraries.
2. Load a high-dimensional dataset.
3. Normalize the dataset.
4. Initialize a Self-Organizing Map.
5. Train the SOM using input data.
6. Find the Best Matching Unit for each data point.
7. Assign cluster labels based on BMU position.
8. Visualize the SOM using a distance map.
9. Plot data distribution on the SOM grid.
10. Analyze clustering and visualization results.

Dataset

Dataset used: **Digits Dataset from Scikit-learn**

The digits dataset contains handwritten digit images. Each data point is an **8 × 8 image**, converted into a feature vector of length **64**. The dataset contains **1797 samples**, **10 classes**, and **64 features**.

Each image represents a digit from 0 to 9.

Code

```
# Experiment: Self-Organizing Map for Clustering and Visualization

# Install MiniSom if not already installed
# !pip install minisom

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_digits
```

```
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import silhouette_score
from minisom import MiniSom

# 1. Load dataset
digits = load_digits()
X = digits.data
y = digits.target

print("Dataset shape:", X.shape)
print("Number of classes:", len(np.unique:))

# 2. Normalize data
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

# 3. Create SOM
som_grid_rows = 10
som_grid_columns = 10

som = MiniSom(
    x=som_grid_rows,
    y=som_grid_columns,
    input_len=X_scaled.shape[1],
    sigma=1.0,
    learning_rate=0.5,
    random_seed=42
)

# 4. Initialize SOM weights
som.random_weights_init(X_scaled)

# 5. Train SOM
som.train_random(data=X_scaled, num_iteration=1000)

# 6. Find Best Matching Unit for each data point
bmu_coordinates = np.array([som.winner(x) for x in X_scaled])

# 7. Convert BMU coordinates into cluster labels
cluster_labels = np.array([
    coord[0] * som_grid_columns + coord[1]
    for coord in bmu_coordinates
])

# 8. Print sample BMU results
print("\nFirst 10 BMU Coordinates:")
print(bmu_coordinates[:10])

print("\nFirst 10 Cluster Labels:")
```

```
print(cluster_labels[:10])

# 9. Calculate silhouette score
# Note: Some SOM nodes may be empty, but labels are still based on BMU positions.
score = silhouette_score(X_scaled, cluster_labels)
print("\nSilhouette Score:", score)

# 10. Visualize SOM distance map
plt.figure(figsize=(8, 7))
plt.pcolor(som.distance_map().T)
plt.colorbar(label="Distance")
plt.title("SOM Distance Map / U-Matrix")
plt.xlabel("SOM X-axis")
plt.ylabel("SOM Y-axis")
plt.show()

# 11. Visualize actual digit classes on SOM grid
plt.figure(figsize=(9, 8))

markers = ['o', 's', 'D', '^', 'v', '<', '>', 'p', '*', 'X']

for i, x in enumerate(X_scaled):
    w = som.winner(x)
    plt.plot(
        w[0] + 0.5,
        w[1] + 0.5,
        markers[y[i]],
        markerfacecolor='None',
        markersize=8,
        markeredgewidth=1.5,
        label=str(y[i]) if i < 10 else ""
    )

plt.xlim([0, som_grid_rows])
plt.ylim([0, som_grid_columns])
plt.title("Digit Classes Mapped on SOM Grid")
plt.xlabel("SOM X-axis")
plt.ylabel("SOM Y-axis")
plt.grid()
plt.show()

# 12. Create result table
result_df = pd.DataFrame({
    "Actual Digit": y,
    "BMU X": bmu_coordinates[:, 0],
    "BMU Y": bmu_coordinates[:, 1],
    "Cluster Label": cluster_labels
})
```

```
print("\nResult Table:")
print(result_df.head(15))
```

Results

Sample expected output:

Parameter	Value
Dataset Samples	1797
Features	64
SOM Grid Size	10 × 10
Number of SOM Nodes	100
Silhouette Score	May vary depending on training

Sample result table:

Actual Digit	BMU X	BMU Y	Cluster Label
0	2	4	24
1	7	3	73
2	5	8	58
3	4	6	46
4	1	9	19

The SOM distance map shows boundaries between clusters. Similar digit images are mapped near each other on the two-dimensional SOM grid.

Outcome of the Experiment

- Self-Organizing Map was successfully implemented using Python.
- High-dimensional handwritten digit data was mapped onto a two-dimensional SOM grid.
- The Best Matching Unit was identified for each input sample.
- Data points were clustered based on their BMU positions.
- The SOM distance map helped visualize cluster boundaries.
- SOM proved useful for clustering and visualization of high-dimensional data.

Relevant Demonstrative Video Link

- Self-Organizing Maps Explained:

- Introduction to Self-Organizing Maps:

- Self-Organizing Feature Map Solved Example:

Other Resource Materials / Web Links

- MiniSom GitHub Documentation: https://github.com/JustGlowing/minisom/
- MiniSom PyPI Page: https://pypi.org/project/MiniSom/
- Scikit-learn Digits Dataset Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html
- Scikit-learn Digits Dataset Example: https://scikit-learn.org/stable/auto_examples/classification/plot_digits_classification.html
- Self-Organizing Maps Explanation: https://builtin.com/articles/self-organizing-maps
- DataCamp SOM Tutorial: https://www.datacamp.com/tutorial/self-organizing-maps

Questions to be asked for Viva Voce:

Q1. Define Self-Organizing Map and its purpose in machine learning.

Q2. Explain the working principle of SOM.

Q3. Describe the role of neurons in a Self-Organizing Map.

Q4. Explain the concept of Best Matching Unit in SOM.

Q5. Discuss the use of SOM for clustering high-dimensional data.

Q6. Explain the importance of neighborhood function in SOM.

Q7. Analyze the role of learning rate in SOM training.

Q8. Compare SOM with traditional clustering methods.

Q9. Describe the visualization output generated by SOM.

Q10. Evaluate the advantages and limitations of SOM in data analysis.

Practical 5

Mapped Course Outcomes - CO3

CO3: Apply appropriate supervised, unsupervised, ensemble, probabilistic, distributed, federated, image generation, and explainable AI techniques to solve real-world machine learning problems.

Manual of the experiment.

Aim

To implement **ensemble learning techniques** on a real-world dataset, evaluate their performance using **cross-validation**, and analyze the impact of different hyperparameters on model accuracy.

Objective

- To understand ensemble learning techniques.
- To implement bagging and boosting models.
- To evaluate models using cross-validation.
- To compare ensemble models on a real-world classification dataset.
- To study the effect of different hyperparameters on model accuracy.
- To identify the best-performing model and parameter combination.

Tools Required

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Scikit-learn

Theory

Ensemble learning is a machine learning technique where multiple models are combined to improve prediction accuracy and stability. Instead of relying on a single model, ensemble methods combine several weak or strong learners to produce a better final result.

Common ensemble learning techniques include:

1. Bagging

Bagging, or Bootstrap Aggregating, trains multiple models independently on different random samples of the dataset. The final output is obtained by averaging or majority voting.

Example: **Random Forest**

Random Forest fits multiple decision trees on different sub-samples of the dataset and uses averaging to improve accuracy and control overfitting.

2. Boosting

Boosting builds models sequentially. Each new model tries to correct the errors made by previous models.

Examples:

- AdaBoost
- Gradient Boosting

Scikit-learn provides ensemble methods such as Random Forest, AdaBoost, and Gradient Boosting through its ensemble module.

3. Cross-Validation

Cross-validation is used to evaluate how well a model performs on unseen data. It divides the dataset into multiple folds, trains the model on some folds, and tests it on the remaining fold. Scikit-learn explains that cross-validation is commonly used in model training workflows, and best parameters can be determined using grid search techniques.

4. Hyperparameter Tuning

Hyperparameters are model settings chosen before training. Examples include:

Model	**Hyperparameters**
Random Forest	n_estimators, max_depth, min_samples_split
AdaBoost	n_estimators, learning_rate
Gradient Boosting	n_estimators, learning_rate, max_depth

GridSearchCV is used to search different hyperparameter combinations using cross-validation. In scikit-learn, the cv parameter determines the cross-validation splitting strategy.

Methodology

1. Import required Python libraries.
2. Load a real-world classification dataset.
3. Split the dataset into input features and target variable.
4. Apply feature scaling where required.
5. Implement ensemble models:
    - Random Forest

- AdaBoost
- Gradient Boosting

6. Evaluate models using 5-fold cross-validation.
7. Compare model accuracy scores.
8. Apply GridSearchCV for hyperparameter tuning.
9. Analyze the impact of different hyperparameters.
10. Display results using tables and graphs.

Dataset

Dataset used: **Breast Cancer Wisconsin Dataset**

The Breast Cancer Wisconsin dataset is a real-world binary classification dataset available in Scikit-learn. It contains **569 samples**, **30 numerical features**, and **2 classes**: malignant and benign.

Dataset details:

Property	**Value**
Samples	569
Features	30
Classes	2
Task Type	Binary Classification

The goal is to classify whether a tumor is **malignant** or **benign** based on numerical medical features.

Code

```
# Experiment: Ensemble Learning with Cross-Validation and Hyperparameter Analysis

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler

from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.ensemble import GradientBoostingClassifier
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# 1. Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# 2. Convert dataset into DataFrame
df = pd.DataFrame(X, columns=data.feature_names)
df["target"] = y

print("First five rows of dataset:")
print(df.head())

print("\nDataset shape:", df.shape)
print("Target classes:", data.target_names)

# 3. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# 4. Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 5. Define ensemble models
models = {
    "Random Forest": RandomForestClassifier(random_state=42),
    "AdaBoost": AdaBoostClassifier(random_state=42),
    "Gradient Boosting": GradientBoostingClassifier(random_state=42)
}

# 6. Cross-validation evaluation
cv_results = []

for name, model in models.items():
    scores = cross_val_score(
        model,
        X_train_scaled,
        y_train,
        cv=5,
        scoring="accuracy"
    )
```

```
    cv_results.append({
        "Model": name,
        "Mean CV Accuracy": scores.mean(),
        "Standard Deviation": scores.std()
    })

cv_results_df = pd.DataFrame(cv_results)

print("\nCross-Validation Results:")
print(cv_results_df)

# 7. Train models and evaluate on test set
test_results = []

for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)

    test_results.append({
        "Model": name,
        "Test Accuracy": accuracy_score(y_test, y_pred)
    })

    print("\n-------------------------------------")
    print("Model:", name)
    print("Confusion Matrix:")
    print(confusion_matrix(y_test, y_pred))
    print("Classification Report:")
    print(classification_report(y_test, y_pred, target_names=data.target_names))

test_results_df = pd.DataFrame(test_results)

print("\nTest Results:")
print(test_results_df)

# 8. Hyperparameter tuning for Random Forest
rf_param_grid = {
    "n_estimators": [50, 100, 150],
    "max_depth": [None, 3, 5, 10],
    "min_samples_split": [2, 5, 10]
}

rf_grid = GridSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_grid=rf_param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)
```

```
rf_grid.fit(X_train_scaled, y_train)

print("\nBest Random Forest Parameters:")
print(rf_grid.best_params_)

print("Best Random Forest CV Accuracy:")
print(rf_grid.best_score_)

# 9. Hyperparameter tuning for AdaBoost
ada_param_grid = {
   "n_estimators": [50, 100, 150],
   "learning_rate": [0.01, 0.1, 0.5, 1.0]
}

ada_grid = GridSearchCV(
   estimator=AdaBoostClassifier(random_state=42),
   param_grid=ada_param_grid,
   cv=5,
   scoring="accuracy",
   n_jobs=-1
)

ada_grid.fit(X_train_scaled, y_train)

print("\nBest AdaBoost Parameters:")
print(ada_grid.best_params_)

print("Best AdaBoost CV Accuracy:")
print(ada_grid.best_score_)

# 10. Hyperparameter tuning for Gradient Boosting
gb_param_grid = {
   "n_estimators": [50, 100, 150],
   "learning_rate": [0.01, 0.1, 0.5],
   "max_depth": [2, 3, 5]
}

gb_grid = GridSearchCV(
   estimator=GradientBoostingClassifier(random_state=42),
   param_grid=gb_param_grid,
   cv=5,
   scoring="accuracy",
   n_jobs=-1
)

gb_grid.fit(X_train_scaled, y_train)

print("\nBest Gradient Boosting Parameters:")
```

```
print(gb_grid.best_params_)

print("Best Gradient Boosting CV Accuracy:")
print(gb_grid.best_score_)

# 11. Compare tuned models
best_models = {
    "Tuned Random Forest": rf_grid.best_estimator_,
    "Tuned AdaBoost": ada_grid.best_estimator_,
    "Tuned Gradient Boosting": gb_grid.best_estimator_
}

tuned_results = []

for name, model in best_models.items():
    y_pred = model.predict(X_test_scaled)

    tuned_results.append({
        "Model": name,
        "Test Accuracy": accuracy_score(y_test, y_pred)
    })

tuned_results_df = pd.DataFrame(tuned_results)

print("\nTuned Model Test Results:")
print(tuned_results_df)

# 12. Plot cross-validation accuracy
plt.figure(figsize=(8, 5))
plt.bar(cv_results_df["Model"], cv_results_df["Mean CV Accuracy"])
plt.xlabel("Model")
plt.ylabel("Mean Cross-Validation Accuracy")
plt.title("Cross-Validation Accuracy of Ensemble Models")
plt.ylim(0.85, 1.0)
plt.xticks(rotation=20)
plt.show()

# 13. Plot tuned model accuracy
plt.figure(figsize=(8, 5))
plt.bar(tuned_results_df["Model"], tuned_results_df["Test Accuracy"])
plt.xlabel("Tuned Model")
plt.ylabel("Test Accuracy")
plt.title("Test Accuracy After Hyperparameter Tuning")
plt.ylim(0.85, 1.0)
plt.xticks(rotation=20)
plt.show()
```

Results

Sample expected cross-validation result:

Model	Mean CV Accuracy	Standard Deviation
Random Forest	0.96	0.02
AdaBoost	0.95	0.02
Gradient Boosting	0.96	0.02

Sample expected hyperparameter tuning result:

Model	Important Hyperparameters	Expected Effect
Random Forest	n_estimators, max_depth	More trees may improve stability; too much depth may overfit
AdaBoost	n_estimators, learning_rate	More estimators and suitable learning rate improve accuracy
Gradient Boosting	n_estimators, learning_rate, max_depth	Controls model complexity and learning speed

The exact output may vary slightly depending on system, train-test split, and library version.

Outcome of the Experiment

- Ensemble learning techniques were successfully implemented.
- Random Forest, AdaBoost, and Gradient Boosting were compared.
- Cross-validation gave a reliable estimate of model performance.
- Hyperparameter tuning improved model selection and accuracy.
- Random Forest performed well due to bagging and averaging of multiple decision trees.
- AdaBoost improved performance by focusing on incorrectly classified samples.
- Gradient Boosting improved performance by sequentially reducing prediction errors.
- The experiment showed that model accuracy depends strongly on hyperparameters such as number of estimators, learning rate, and tree depth.

Relevant Demonstrative Video Link

- Ensemble Methods in Scikit-Learn:

- GridSearchCV Cross-Validation in Scikit-learn:

- Breast Cancer Classification using Machine Learning:

Other Resource Materials / Web Links

- Scikit-learn Ensemble Methods User Guide: https://scikit-learn.org/stable/modules/ensemble.html
- RandomForestClassifier Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html
- Scikit-learn Cross-Validation User Guide: https://scikit-learn.org/stable/modules/cross_validation.html
- GridSearchCV Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html
- Breast Cancer Dataset Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html

Questions to be asked for Viva Voce:

Q1. Define ensemble learning and mention its importance in machine learning.

Q2. Explain the concept of bagging with a suitable example.

Q3. Explain the concept of boosting with a suitable example.

Q4. Differentiate Random Forest, AdaBoost and Gradient Boosting.

Q5. Describe the role of cross-validation in model evaluation.

Q6. Identify important hyperparameters in ensemble models.

Q7. Analyze the impact of number of estimators on model accuracy.

Q8. Discuss the effect of tree depth on ensemble model performance.

Q9. Compare ensemble learning with a single machine learning model.

Q10. Evaluate the advantages of ensemble learning on real-world datasets.

Practical 6

Mapped Course Outcomes - CO4

CO4: Analyse and evaluate machine learning models using suitable performance parameters and cross-validation to identify the best-suited techniques for specific applications.

Manual of the experiment.

Aim

To implement a **Deep Belief Network (DBN)** on a simple dataset and visualize the network learned features.

Objective

- To understand the concept of Deep Belief Network.
- To implement DBN using Python.
- To use Restricted Boltzmann Machines as feature-learning layers.
- To apply DBN on a simple image dataset.
- To visualize the features learned by the network.
- To evaluate the performance of the DBN-based model.

Tools Required

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Scikit-learn

Theory

A **Deep Belief Network (DBN)** is a deep learning model made by stacking multiple **Restricted Boltzmann Machines (RBMs)**. It is mainly used for unsupervised feature learning and classification.

An **RBM** is a neural network with two layers:

- Visible layer
- Hidden layer

The visible layer receives input data, and the hidden layer learns useful features from that data. In a DBN, the output features learned by one RBM are passed to the next RBM layer.

DBN training usually happens in two stages:

1. **Unsupervised pre-training**
   RBM layers learn features from input data.
2. **Supervised fine-tuning or classification**
   A classifier such as Logistic Regression is trained on the learned features.

Scikit-learn provides BernoulliRBM, which can be used for nonlinear feature extraction. Its documentation also provides an example of RBM features for digit classification.

Methodology

1. Import required Python libraries.
2. Load the Digits dataset.
3. Normalize image pixel values.
4. Split the data into training and testing sets.
5. Create a DBN-like model using stacked RBM layers.
6. Train RBM layers one by one.
7. Extract learned features from RBM layers.
8. Train a Logistic Regression classifier on extracted features.
9. Evaluate classification accuracy.
10. Visualize the features learned by the first RBM layer.

Dataset

Dataset used: **Digits Dataset from Scikit-learn**

The Digits dataset contains handwritten digit images from 0 to 9. Each image is of size **8 × 8 pixels**, which gives **64 input features** per sample.

Dataset details:

Property	**Value**
Samples	1797
Image Size	8 × 8
Features	64
Classes	10
Task	Digit classification

The Scikit-learn digits dataset is commonly used for classification experiments and contains handwritten digit images.

Code

```
# Experiment: Deep Belief Network using Stacked RBM Layers

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.neural_network import BernoulliRBM
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# 1. Load dataset
digits = load_digits()
X = digits.data
y = digits.target

print("Dataset shape:", X.shape)
print("Target classes:", np.unique:)

# 2. Normalize pixel values between 0 and 1
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

# 3. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# 4. Define first RBM layer
rbm1 = BernoulliRBM(
    n_components=64,
    learning_rate=0.05,
    n_iter=30,
    batch_size=10,
    random_state=42,
    verbose=True
```

```
)

# 5. Define second RBM layer
rbm2 = BernoulliRBM(
    n_components=32,
    learning_rate=0.05,
    n_iter=30,
    batch_size=10,
    random_state=42,
    verbose=True
)

# 6. Train first RBM layer
print("\nTraining First RBM Layer...")
rbm1.fit(X_train)

# 7. Transform data using first RBM
X_train_rbm1 = rbm1.transform(X_train)
X_test_rbm1 = rbm1.transform(X_test)

# 8. Train second RBM layer
print("\nTraining Second RBM Layer...")
rbm2.fit(X_train_rbm1)

# 9. Transform data using second RBM
X_train_features = rbm2.transform(X_train_rbm1)
X_test_features = rbm2.transform(X_test_rbm1)

# 10. Train classifier on DBN learned features
classifier = LogisticRegression(max_iter=5000)
classifier.fit(X_train_features, y_train)

# 11. Predict test data
y_pred = classifier.predict(X_test_features)

# 12. Evaluate performance
accuracy = accuracy_score(y_test, y_pred)

print("\nDBN Classification Accuracy:", accuracy)

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# 13. Visualize original digit images
plt.figure(figsize=(8, 4))
for i in range(10):
```

```
    plt.subplot(2, 5, i + 1)
    plt.imshow(X[i].reshape(8, 8), cmap="gray")
    plt.title("Digit: " + str(y[i]))
    plt.axis("off")

plt.suptitle("Sample Digit Images")
plt.show()

# 14. Visualize learned features from first RBM layer
plt.figure(figsize=(10, 10))

for i, component in enumerate(rbm1.components_[:64]):
    plt.subplot(8, 8, i + 1)
    plt.imshow(component.reshape(8, 8), cmap="gray")
    plt.axis("off")

plt.suptitle("Features Learned by First RBM Layer")
plt.show()

# 15. Compare with Logistic Regression on raw data
raw_classifier = LogisticRegression(max_iter=5000)
raw_classifier.fit(X_train, y_train)

raw_pred = raw_classifier.predict(X_test)
raw_accuracy = accuracy_score(y_test, raw_pred)

print("\nLogistic Regression Accuracy on Raw Data:", raw_accuracy)

# 16. Result comparison table
results = pd.DataFrame({
    "Model": [
        "Logistic Regression on Raw Data",
        "DBN Features + Logistic Regression"
    ],
    "Accuracy": [
        raw_accuracy,
        accuracy
    ]
})

print("\nResult Comparison:")
print(results)
```

Results

Sample expected result:

Model	**Accuracy**

Logistic Regression on Raw Data	0.95
DBN Features + Logistic Regression	0.85–0.95

The exact accuracy may vary depending on:

- Number of RBM components
- Learning rate
- Number of iterations
- Train-test split
- Random initialization

The learned feature visualization shows patterns captured by the RBM from digit images. These features may look like strokes, edges, or small digit-like patterns.

Outcome of the Experiment

- A Deep Belief Network was implemented using stacked RBM layers.
- The Digits dataset was used for classification.
- RBM layers learned hidden features from handwritten digit images.
- The learned features were passed to a Logistic Regression classifier.
- The first RBM layer's learned features were visualized.
- The experiment showed that DBNs can learn useful representations from high-dimensional image data.
- It also showed how unsupervised feature learning can support supervised classification.

Relevant Demonstrative Video Link

- Restricted Boltzmann Machine Tutorial:
- Deep Belief Network Pre-training:
- DBN using Scikit-learn BernoulliRBM:

Other Resource Materials / Web Links

- Scikit-learn BernoulliRBM Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.BernoulliRBM.html

- Scikit-learn RBM Features for Digit Classification: https://scikit-learn.org/stable/auto_examples/neural_networks/plot_rbm_logistic_classification.html
- DBN and RBM Tutorial Survey Paper: https://arxiv.org/abs/2107.12521
- PyImageSearch RBM on MNIST using Python: https://pyimagesearch.com/2014/06/23/applying-deep-learning-rbm-mnist-using-python/
- Deep Belief Network Explanation: https://www.geeksforgeeks.org/deep-learning/deep-belief-network-dbn-in-deep-learning/

Questions to be asked for Viva Voce:

Q1. Define Deep Belief Network and mention its use in deep learning.

Q2. Explain the architecture of a Deep Belief Network.

Q3. Describe the role of Restricted Boltzmann Machines in DBN.

Q4. Explain layer-wise training in Deep Belief Networks.

Q5. Discuss the difference between supervised and unsupervised learning in DBN.

Q6. Identify the role of hidden layers in feature learning.

Q7. Explain the need of feature visualization in DBN.

Q8. Analyze the learned features generated by a DBN model.

Q9. Compare DBN with traditional neural networks.

Q10. Evaluate the applications and limitations of Deep Belief Networks.

Practical 7

Mapped Course Outcomes - CO4

CO4: Analyse and evaluate machine learning models using suitable performance parameters and cross-validation to identify the best-suited techniques for specific applications.

Manual of the experiment.

Aim

To implement **Gibbs Sampling** in Python and understand how it is used to generate samples from a probability distribution when direct sampling is difficult.

Objective

- To understand the concept of sampling from probability distributions.
- To study the working of Gibbs Sampling.
- To implement Gibbs Sampling using Python.
- To generate samples from a joint probability distribution.
- To visualize the sampled data.
- To understand how Gibbs Sampling is useful in probabilistic machine learning.

Tools Required

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Seaborn, optional
- SciPy, optional

Theory

Gibbs Sampling is a Markov Chain Monte Carlo method used to generate samples from a complex probability distribution.

In many real-world problems, direct sampling from a joint probability distribution is difficult. Gibbs Sampling solves this problem by sampling one variable at a time from its conditional distribution while keeping other variables fixed.

Suppose we have two random variables and . Instead of directly sampling from the joint distribution:

After many iterations, the generated samples approximate the original joint distribution.

Basic Idea

If direct sampling from is difficult, but sampling from conditional distributions and is easy, then Gibbs Sampling can be used.

Gibbs Sampling Steps

1. Initialize variables with random values.
2. Sample one variable from its conditional distribution.
3. Sample the next variable using the updated value of the previous variable.
4. Repeat the process many times.
5. After enough iterations, the samples represent the target distribution.

Methodology

1. Import required Python libraries.
2. Define the target joint distribution.
3. Initialize random values for variables.
4. Define conditional sampling equations.
5. Run Gibbs Sampling for a fixed number of iterations.
6. Store generated samples.
7. Remove initial burn-in samples.
8. Visualize generated samples using scatter plot and histogram.
9. Analyse the sampled distribution.

Dataset

No external dataset is required for this experiment.

A synthetic probability distribution is used.

In this experiment, we generate samples from a **bivariate normal distribution** with two variables. The distribution has:

Parameter	**Value**
Mean of X	0
Mean of Y	0

Standard Deviation of X	1
Standard Deviation of Y	1
Correlation	0.8

This dataset is generated through Gibbs Sampling.

Code

```
# Experiment: Gibbs Sampling using Python

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 1. Set random seed for reproducibility
np.random.seed(42)

# 2. Define parameters for bivariate normal distribution
mu_x = 0
mu_y = 0
sigma_x = 1
sigma_y = 1
rho = 0.8

# 3. Define number of iterations
num_samples = 10000
burn_in = 1000

# 4. Initialize arrays to store samples
x_samples = np.zeros(num_samples)
y_samples = np.zeros(num_samples)

# 5. Initial values
x_samples[0] = 0
y_samples[0] = 0

# 6. Gibbs Sampling
for i in range(1, num_samples):

    # Sample X from conditional distribution P(X | Y)
    mean_x_given_y = mu_x + rho * (sigma_x / sigma_y) * (y_samples[i-1] - mu_y)
    std_x_given_y = sigma_x * np.sqrt(1 - rho**2)

    x_samples[i] = np.random.normal(mean_x_given_y, std_x_given_y)

    # Sample Y from conditional distribution P(Y | X)
    mean_y_given_x = mu_y + rho * (sigma_y / sigma_x) * (x_samples[i] - mu_x)
```

```
    std_y_given_x = sigma_y * np.sqrt(1 - rho**2)

    y_samples[i] = np.random.normal(mean_y_given_x, std_y_given_x)

# 7. Remove burn-in samples
x_final = x_samples[burn_in:]
y_final = y_samples[burn_in:]

# 8. Create DataFrame
samples_df = pd.DataFrame({
    "X": x_final,
    "Y": y_final
})

print("First five generated samples:")
print(samples_df.head())

print("\nMean of X:", np.mean(x_final))
print("Mean of Y:", np.mean(y_final))
print("Correlation between X and Y:", np.corrcoef(x_final, y_final)[0, 1])

# 9. Scatter plot of generated samples
plt.figure(figsize=(7, 5))
plt.scatter(x_final, y_final, alpha=0.3, s=10)
plt.xlabel("X Samples")
plt.ylabel("Y Samples")
plt.title("Gibbs Sampling: Generated Samples from Bivariate Normal Distribution")
plt.show()

# 10. Histogram of X samples
plt.figure(figsize=(7, 4))
plt.hist(x_final, bins=40, density=True, alpha=0.7)
plt.xlabel("X")
plt.ylabel("Density")
plt.title("Histogram of X Samples")
plt.show()

# 11. Histogram of Y samples
plt.figure(figsize=(7, 4))
plt.hist(y_final, bins=40, density=True, alpha=0.7)
plt.xlabel("Y")
plt.ylabel("Density")
plt.title("Histogram of Y Samples")
plt.show()

# 12. Trace plot for X
plt.figure(figsize=(8, 4))
plt.plot(x_samples[:1000])
plt.xlabel("Iteration")
```

```
plt.ylabel("X Value")
plt.title("Trace Plot of X Samples")
plt.show()

# 13. Trace plot for Y
plt.figure(figsize=(8, 4))
plt.plot(y_samples[:1000])
plt.xlabel("Iteration")
plt.ylabel("Y Value")
plt.title("Trace Plot of Y Samples")
plt.show()
```

Results

Sample expected output:

Output	**Expected Observation**
Mean of X	Close to 0
Mean of Y	Close to 0
Correlation	Close to 0.8
Scatter Plot	Elliptical cloud of points
Histogram	Bell-shaped distribution
Trace Plot	Random movement around mean

Sample generated values may look like:

X	**Y**
0.298	0.155
0.512	0.738
-0.214	-0.386
1.025	0.891
-0.604	-0.552

The exact values may change because sampling is random.

Outcome of the Experiment

- Gibbs Sampling was successfully implemented using Python.
- Samples were generated from a bivariate normal distribution.
- Conditional distributions were used instead of direct sampling from the joint distribution.

- Initial burn-in samples were removed to improve sample quality.
- Scatter plot, histograms, and trace plots were used to visualize the generated samples.
- The generated samples approximated the target distribution.
- The experiment demonstrated that Gibbs Sampling is useful when direct sampling from a joint distribution is difficult but conditional sampling is possible.

Relevant Demonstrative Video Link

- Gibbs Sampling Clearly Explained:
- Markov Chain Monte Carlo and Gibbs Sampling:
- Gibbs Sampling Tutorial:

Other Resource Materials / Web Links

- Gibbs Sampling Explanation: https://en.wikipedia.org/wiki/Gibbs_sampling
- Markov Chain Monte Carlo Overview: https://en.wikipedia.org/wiki/Markov_chain_Monte_Carlo
- Gibbs Sampling in Machine Learning: https://www.geeksforgeeks.org/machine-learning/gibbs-sampling-in-machine-learning/
- NumPy Random Normal Documentation: https://numpy.org/doc/stable/reference/random/generated/numpy.random.normal.html
- SciPy Statistics Documentation: https://docs.scipy.org/doc/scipy/reference/stats.html

Questions to be asked for Viva Voce:

Q1. Define Gibbs Sampling and mention its use in probability distribution sampling.
Q2. Explain the need of Gibbs Sampling when direct sampling is difficult.
Q3. Describe the concept of conditional probability in Gibbs Sampling.
Q4. Explain the step-by-step working of Gibbs Sampling.
Q5. Discuss the role of Markov Chain Monte Carlo in Gibbs Sampling.
Q6. Identify suitable probability distributions for Gibbs Sampling.
Q7. Analyze the convergence behavior of Gibbs Sampling.
Q8. Explain the importance of initial values in Gibbs Sampling.
Q9. Compare Gibbs Sampling with direct random sampling.

Practical 8

Mapped Course Outcomes - CO4

CO4: Analyse and evaluate machine learning models using suitable performance parameters and cross-validation to identify the best-suited techniques for specific applications.

Manual of the experiment.

Aim

To implement a **Hidden Markov Model (HMM)** to predict sequences in time-series data such as weather forecasting or speech recognition.

Objective

- To understand the concept of Hidden Markov Models.
- To implement HMM using Python.
- To model sequential/time-series data using hidden states and observations.
- To predict hidden state sequences from observed data.
- To understand the use of transition and emission probabilities.
- To visualize and analyse the predicted sequence.

Tools Required

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- hmmlearn library

Theory

A **Hidden Markov Model (HMM)** is a probabilistic model used for sequential data. It assumes that the system has some **hidden states** that cannot be directly observed, but they generate visible observations.

For example, in weather forecasting:

- Hidden states: Sunny, Rainy
- Observations: Walk, Shop, Clean

In speech recognition:

- Hidden states: phonemes or speech units
- Observations: audio signal features

An HMM is based on the **Markov property**, which means the current state depends only on the previous state, not on the entire history.

An HMM mainly consists of:

Component	**Meaning**
Hidden States	Actual states of the system that are not directly visible
Observations	Visible output generated by hidden states
Start Probability	Probability of starting in each hidden state
Transition Probability	Probability of moving from one hidden state to another
Emission Probability	Probability of observing output from a hidden state

The hmmlearn library implements Hidden Markov Models and describes HMMs as generative probabilistic models where observable variables are generated by internal hidden states; state transitions follow a first-order Markov chain.

Methodology

1. Import required Python libraries.
2. Define hidden states and observation symbols.
3. Define start probability, transition probability, and emission probability.
4. Create a Hidden Markov Model.
5. Provide an observation sequence.
6. Use the model to predict the most likely hidden state sequence.
7. Display predicted states.
8. Visualize the predicted hidden states.
9. Analyze the output.

Dataset

No external dataset is required for this experiment.

A simple synthetic weather dataset is used.

Hidden States

State Code	**Hidden State**
0	Rainy

1	Sunny

Observations

Observation Code	Observation
0	Walk
1	Shop
2	Clean

Example Observation Sequence

Walk → Shop → Clean → Walk → Walk → Shop → Clean

The HMM predicts the most likely weather sequence that produced these observations.

Code

```
# Experiment: Hidden Markov Model for Weather Sequence Prediction

# Install hmmlearn if not installed
# !pip install hmmlearn

import numpy as np
import matplotlib.pyplot as plt
from hmmlearn import hmm

# 1. Define hidden states
states = ["Rainy", "Sunny"]
n_states = len(states)

# 2. Define observations
observations = ["Walk", "Shop", "Clean"]
n_observations = len(observations)

# 3. Create HMM model
model = hmm.CategoricalHMM(n_components=n_states, random_state=42)

# 4. Define start probabilities
# Probability of starting with Rainy or Sunny
model.startprob_ = np.array([0.6, 0.4])

# 5. Define transition probabilities
# Rows: current state, Columns: next state
# Rainy -> Rainy, Rainy -> Sunny
# Sunny -> Rainy, Sunny -> Sunny
model.transmat_ = np.array([
    [0.7, 0.3],
    [0.4, 0.6]
```

```
])

# 6. Define emission probabilities
# Rows: hidden states, Columns: observations
# Rainy: Walk, Shop, Clean
# Sunny: Walk, Shop, Clean
model.emissionprob_ = np.array([
    [0.1, 0.4, 0.5],
    [0.6, 0.3, 0.1]
])

# 7. Define observation sequence
# Walk, Shop, Clean, Walk, Walk, Shop, Clean
observation_sequence = np.array([[0], [1], [2], [0], [0], [1], [2]])

# 8. Predict most likely hidden state sequence
log_probability, hidden_state_sequence = model.decode(
    observation_sequence,
    algorithm="viterbi"
)

# 9. Display results
print("Observation Sequence:")
for obs in observation_sequence:
    print(observations[obs[0]], end=" ")

print("\n\nPredicted Hidden State Sequence:")
for state in hidden_state_sequence:
    print(states[state], end=" ")

print("\n\nLog Probability of Sequence:", log_probability)

# 10. Create a readable table
print("\n\nObservation and Predicted Weather:")
print("-----------------------------------")
for i in range(len(observation_sequence)):
    obs_name = observations[observation_sequence[i][0]]
    state_name = states[hidden_state_sequence[i]]
    print(f"Time {i+1}: Observation = {obs_name}, Predicted Weather = {state_name}")

# 11. Visualize predicted states
plt.figure(figsize=(8, 4))
plt.plot(hidden_state_sequence, marker="o")
plt.yticks([0, 1], states)
plt.xticks(range(len(observation_sequence)),
           [observations[o[0]] for o in observation_sequence])
plt.xlabel("Observation Sequence")
plt.ylabel("Predicted Hidden State")
plt.title("Hidden Markov Model: Predicted Weather Sequence")
```

```
plt.grid(True)
plt.show()

# 12. Generate sample sequence from model
generated_observations, generated_states = model.sample(10)

print("\nGenerated Observation Sequence:")
for obs in generated_observations:
    print(observations[obs[0]], end=" ")

print("\n\nGenerated Hidden State Sequence:")
for state in generated_states:
    print(states[state], end=" ")
```

Results

Sample expected output:

Observation Sequence:
Walk Shop Clean Walk Walk Shop Clean

Predicted Hidden State Sequence:
Sunny Rainy Rainy Sunny Sunny Rainy Rainy

Log Probability of Sequence: -8.65

Sample result table:

Time	Observation	Predicted Hidden State
1	Walk	Sunny
2	Shop	Rainy
3	Clean	Rainy
4	Walk	Sunny
5	Walk	Sunny
6	Shop	Rainy
7	Clean	Rainy

The exact generated sample sequence may change because random sampling is involved.

Outcome of the Experiment

- A Hidden Markov Model was successfully implemented using Python.
- Hidden states and observations were defined for a weather prediction example.

- Start probability, transition probability, and emission probability were used to build the model.
- The Viterbi algorithm was used to find the most likely hidden state sequence.
- The model predicted weather states based on observed activities.
- The experiment showed how HMMs can be used for sequence prediction in time-series problems such as weather forecasting and speech recognition.

Relevant Demonstrative Video Link

- Hidden Markov Models Clearly Explained:
- Hidden Markov Model and Viterbi Algorithm:
- Hidden Markov Model with Python:

Other Resource Materials / Web Links

- hmmlearn Tutorial: https://hmmlearn.readthedocs.io/en/latest/tutorial.html
- hmmlearn API Reference: https://hmmlearn.readthedocs.io/en/latest/api.html
- hmmlearn Categorical / Multinomial HMM Example: https://hmmlearn.readthedocs.io/en/latest/auto_examples/plot_multinomial_hmm.html
- hmmlearn GitHub Repository: https://github.com/hmmlearn/hmmlearn
- Hidden Markov Model Explanation: https://en.wikipedia.org/wiki/Hidden_Markov_model

Questions to be asked for Viva Voce:

Q1. Define Hidden Markov Model and mention its use in sequence prediction.

Q2. Explain the components of an HMM.

Q3. Describe hidden states and observed states in HMM.

Q4. Explain transition probability in Hidden Markov Models.

Q5. Explain emission probability with a suitable example.

Practical 9

Mapped Course Outcomes - CO4

CO4: Analyse and evaluate machine learning models using suitable performance parameters and cross-validation to identify the best-suited techniques for specific applications.

Manual of the experiment.

Aim

To implement a **Federated Learning system** using a machine learning model.

Objective

- To understand the basic concept of Federated Learning.
- To simulate multiple clients using one dataset.
- To train local machine learning models on different client data.
- To combine local model outputs into a global prediction system.
- To evaluate the performance of the federated model.
- To understand how Federated Learning supports privacy-preserving machine learning.

Tools Required

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Scikit-learn
- SciPy

Theory

Federated Learning is a machine learning approach where a model is trained across multiple clients or devices without directly sharing their raw data.

In normal centralized machine learning, all data is collected in one place and then used to train a model. This may create privacy and security problems.

In Federated Learning, data stays with each client. Each client trains a local model on its own data. The local models or their predictions are then combined to form a global learning system.

Example:

- Hospital 1 trains a model on its patient data.
- Hospital 2 trains a model on its patient data.
- Hospital 3 trains a model on its patient data.
- Patient data is not shared.
- The models are combined to improve prediction.

Basic structure:

Client 1 Data → Local Model 1
Client 2 Data → Local Model 2
Client 3 Data → Local Model 3
Client 4 Data → Local Model 4

Local Model Outputs → Aggregation → Global Prediction

In this experiment, Federated Learning is simulated by dividing a dataset into multiple clients. Each client trains a local Logistic Regression model. The predictions from local models are combined using **majority voting**.

Methodology

1. Import required Python libraries.
2. Load a real-world classification dataset.
3. Split the dataset into training and testing sets.
4. Standardize the feature values.
5. Divide the training data into multiple client datasets.
6. Train a local model for each client.
7. Evaluate each local model on the common test set.
8. Collect predictions from all local models.
9. Combine predictions using majority voting.
10. Evaluate the final federated learning system.
11. Compare local model accuracy with global federated accuracy.

Dataset

Dataset used: **Breast Cancer Wisconsin Dataset**

This dataset is available in Scikit-learn and is used for binary classification.

Dataset details:

Property	Description
Dataset name	Breast Cancer Wisconsin Dataset
Number of samples	569
Number of features	30
Number of classes	2
Classes	Malignant, Benign
Task	Tumor classification

The target is to classify whether a tumor is **malignant** or **benign** based on numerical medical features.

Code

```
# Experiment: Federated Learning System using a Machine Learning Model

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from scipy.stats import mode

# 1. Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

print("Dataset Shape:", X.shape)
print("Target Classes:", data.target_names)

# 2. Create DataFrame
df = pd.DataFrame(X, columns=data.feature_names)
df["target"] = y

print("\nFirst five rows of dataset:")
print(df.head())

# 3. Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
```

```
    test_size=0.2,
    random_state=42,
    stratify=y
)

# 4. Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 5. Define number of clients
num_clients = 5

# 6. Divide training data among clients
X_clients = np.array_split(X_train_scaled, num_clients)
y_clients = np.array_split(y_train, num_clients)

# 7. Train local models
client_models = []
client_results = []

for i in range(num_clients):
    local_model = LogisticRegression(max_iter=1000)

    # Train local model on client data
    local_model.fit(X_clients[i], y_clients[i])

    # Store trained model
    client_models.append(local_model)

    # Evaluate local model
    y_pred_client = local_model.predict(X_test_scaled)
    client_accuracy = accuracy_score(y_test, y_pred_client)

    client_results.append({
        "Client": f"Client {i + 1}",
        "Training Samples": len(X_clients[i]),
        "Accuracy": client_accuracy
    })

    print(f"\nClient {i + 1} Accuracy: {client_accuracy:.4f}")

# 8. Collect predictions from all client models
all_client_predictions = []

for model in client_models:
    predictions = model.predict(X_test_scaled)
    all_client_predictions.append(predictions)
```

```
all_client_predictions = np.array(all_client_predictions)

# 9. Combine predictions using majority voting
global_predictions = mode(
    all_client_predictions,
    axis=0,
    keepdims=False
).mode

# 10. Evaluate federated/global system
global_accuracy = accuracy_score(y_test, global_predictions)

print("\nFederated Global Model Accuracy:", global_accuracy)

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, global_predictions))

print("\nClassification Report:")
print(classification_report(y_test, global_predictions, target_names=data.target_names))

# 11. Create result table
results_df = pd.DataFrame(client_results)

global_row = pd.DataFrame([{
    "Client": "Federated Global Model",
    "Training Samples": len(X_train_scaled),
    "Accuracy": global_accuracy
}])

final_results = pd.concat([results_df, global_row], ignore_index=True)

print("\nFinal Accuracy Comparison:")
print(final_results)

# 12. Plot accuracy comparison
plt.figure(figsize=(9, 5))
plt.bar(final_results["Client"], final_results["Accuracy"])
plt.xlabel("Model")
plt.ylabel("Accuracy")
plt.title("Client Models vs Federated Global Model Accuracy")
plt.ylim(0.80, 1.00)
plt.xticks(rotation=25)
plt.show()
```

Results

Sample expected output:

Model	Training Samples	Accuracy
Client 1	91	0.95
Client 2	91	0.96
Client 3	91	0.94
Client 4	91	0.96
Client 5	91	0.95
Federated Global Model	455	0.96

Sample confusion matrix:

	Predicted Malignant	Predicted Benign
Actual Malignant	39	3
Actual Benign	2	70

The exact values may vary depending on train-test split and client distribution.

Outcome of the Experiment

- A Federated Learning system was successfully implemented.
- The dataset was divided into multiple clients.
- Each client trained a separate local machine learning model.
- Raw client data was not shared between clients.
- Predictions from local client models were combined using majority voting.
- The federated global model produced stable classification performance.
- The experiment demonstrated the basic idea of privacy-preserving distributed machine learning.

Relevant Demonstrative Video Link

- Federated Learning Explained:
- Federated Learning Tutorial:
- Federated Learning with Python:

Other Resource Materials / Web Links

- TensorFlow Federated Official Website: https://www.tensorflow.org/federated

- TensorFlow Federated Tutorials: https://www.tensorflow.org/federated/tutorials/tutorials_overview
- Federated Learning by Google AI: https://blog.google/technology/ai/federated-learning-collaborative-machine-learning-without-centralized-training-data/
- Scikit-learn Breast Cancer Dataset Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html
- Scikit-learn Logistic Regression Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

Questions to be asked for Viva Voce:

Q1. Define Federated Learning and mention its main objective.

Q2. Explain the need of Federated Learning in privacy-preserving machine learning.

Q3. Describe the architecture of a Federated Learning system.

Q4. Explain the role of server and clients in Federated Learning.

Q5. Discuss the training process in Federated Learning.

Q6. Explain model update sharing in Federated Learning.

Q7. Identify the importance of data privacy in Federated Learning.

Q8. Compare Federated Learning with centralized machine learning.

Q9. Analyze the challenges of communication in Federated Learning.

Q10. Evaluate the applications of Federated Learning in healthcare and banking.

Practical 10

Mapped Course Outcomes - CO4

CO4: Analyse and evaluate machine learning models using suitable performance parameters and cross-validation to identify the best-suited techniques for specific applications.

Manual of the experiment.

Aim

To implement a **Federated Learning** experiment by dividing one dataset into multiple clients, training models separately, and combining their predictions.

Objective

- To understand the concept of Federated Learning.
- To divide a dataset into multiple client datasets.
- To train separate machine learning models on each client.
- To combine predictions from different client models.
- To compare client-wise model performance with combined prediction performance.
- To understand how Federated Learning helps preserve data privacy.

Tools Required

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Scikit-learn

Theory

Federated Learning is a machine learning technique where multiple clients train models locally using their own data, and the learned information is combined without directly sharing raw data.

In traditional machine learning, all data is collected in one central server. This may create privacy and security issues.

In Federated Learning:

- Data remains with each client.

- Each client trains a local model.
- Only model updates or predictions are shared.
- A central system combines the results.

For example, hospitals may train models on patient data locally. They do not need to share patient records with each other. Instead, their model outputs or updates can be combined to create a better global model.

Federated Learning Concept

Client 1 Data → Local Model 1
Client 2 Data → Local Model 2
Client 3 Data → Local Model 3
Client 4 Data → Local Model 4

Local Model Predictions → Combined Final Prediction

In this experiment, we simulate Federated Learning by dividing one dataset into multiple clients.

Each client trains a separate model. Then, predictions from all client models are combined using majority voting.

Methodology

1. Import required Python libraries.
2. Load a classification dataset.
3. Split the dataset into training and testing data.
4. Divide training data into multiple clients.
5. Train one local model for each client.
6. Evaluate each client model separately.
7. Generate predictions from all client models.
8. Combine predictions using majority voting.
9. Evaluate the combined global prediction.
10. Compare client-wise accuracy and global accuracy.

Dataset

Dataset used: **Breast Cancer Wisconsin Dataset**

The Breast Cancer Wisconsin dataset is a binary classification dataset available in Scikit-learn.

Dataset details:

Property	Value
Samples	569
Features	30
Classes	2
Class Names	Malignant, Benign
Task Type	Binary Classification

The target is to classify whether a tumor is **malignant** or **benign** based on medical features.

Code

```
# Experiment: Federated Learning Simulation using Multiple Clients

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from scipy.stats import mode

# 1. Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

print("Dataset shape:", X.shape)
print("Target classes:", data.target_names)

# 2. Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# 3. Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)

# 4. Divide training data into multiple clients
num_clients = 4

X_client_splits = np.array_split(X_train_scaled, num_clients)
y_client_splits = np.array_split(y_train, num_clients)

# 5. Train local model on each client
client_models = []
client_accuracies = []

for i in range(num_clients):
    model = LogisticRegression(max_iter=1000)

    model.fit(X_client_splits[i], y_client_splits[i])

    client_models.append(model)

    y_pred_client = model.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred_client)
    client_accuracies.append(acc)

    print(f"\nClient {i+1} Model Accuracy: {acc:.4f}")

# 6. Collect predictions from all client models
all_predictions = []

for model in client_models:
    pred = model.predict(X_test_scaled)
    all_predictions.append(pred)

all_predictions = np.array(all_predictions)

# 7. Combine predictions using majority voting
global_predictions = mode(all_predictions, axis=0, keepdims=False).mode

# 8. Evaluate combined global model
global_accuracy = accuracy_score(y_test, global_predictions)

print("\nGlobal Combined Model Accuracy:", global_accuracy)

print("\nConfusion Matrix of Global Model:")
print(confusion_matrix(y_test, global_predictions))

print("\nClassification Report of Global Model:")
print(classification_report(y_test, global_predictions, target_names=data.target_names))

# 9. Compare client and global accuracy
```

```
results = pd.DataFrame({
    "Model": [
        "Client 1",
        "Client 2",
        "Client 3",
        "Client 4",
        "Global Combined Model"
    ],
    "Accuracy": client_accuracies + [global_accuracy]
})

print("\nAccuracy Comparison:")
print(results)

# 10. Plot accuracy comparison
plt.figure(figsize=(8, 5))
plt.bar(results["Model"], results["Accuracy"])
plt.xlabel("Model")
plt.ylabel("Accuracy")
plt.title("Federated Learning: Client Models vs Global Combined Model")
plt.ylim(0.8, 1.0)
plt.xticks(rotation=25)
plt.show()
```

Results

Sample expected output:

Model	**Accuracy**
Client 1	0.95
Client 2	0.96
Client 3	0.94
Client 4	0.96
Global Combined Model	0.96

The exact accuracy may change depending on train-test split and client data distribution.

The global combined model generally performs better or more stable than some individual client models because it combines knowledge from multiple local models.

Outcome of the Experiment

- Federated Learning was successfully simulated using Python.
- A single dataset was divided into multiple clients.
- Each client trained a separate local model.

- Raw training data was not shared between clients.
- Predictions from client models were combined using majority voting.
- The combined global prediction showed stable performance.
- The experiment demonstrated the basic idea of Federated Learning, where learning happens locally and results are combined centrally.

Relevant Demonstrative Video Link

- Federated Learning Explained:
- Federated Learning Tutorial:
- Federated Learning with Python:

Other Resource Materials / Web Links

- TensorFlow Federated Official Website: https://www.tensorflow.org/federated
- TensorFlow Federated Tutorials: https://www.tensorflow.org/federated/tutorials/tutorials_overview
- Federated Learning Explanation by Google AI: https://ai.googleblog.com/2017/04/federated-learning-collaborative.html
- Scikit-learn Breast Cancer Dataset Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html
- Scikit-learn Logistic Regression Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

Questions to be asked for Viva Voce:

Q1. Explain the concept of dividing one dataset into multiple clients.

Q2. Describe the need of client-wise data partitioning in Federated Learning.

Q3. Explain local model training on each client.

Q4. Discuss the process of combining predictions from multiple clients.

Q5. Analyse the difference between local training and global learning.

Q6. Explain the role of aggregation in Federated Learning.

Q7. Identify the challenges of non-IID data in client-based learning.

Practical 11

Mapped Course Outcomes - CO5

CO5 (Create): Design and implement optimized machine learning solutions by integrating advanced supervised algorithms, unsupervised learning methods, ensemble learning, federated learning, image generation, and explainable AI for complex predictive and decision-making problems.

Manual of the experiment.

Aim

To implement a simple **image segmentation** experiment using Python and OpenCV to separate an object or region of interest from an image.

Objective

- To understand the concept of image segmentation.
- To read and process an image using OpenCV.
- To convert an image into grayscale.
- To apply thresholding for separating foreground and background.
- To detect contours of the segmented object.
- To extract and visualize the region of interest.
- To understand practical use of segmentation in computer vision.

Tools Required

- Python
- Jupyter Notebook / Google Colab
- OpenCV
- NumPy
- Matplotlib

Theory

Image segmentation is a computer vision technique used to divide an image into meaningful parts or regions. The main purpose of segmentation is to separate important objects from the background.

In this experiment, simple segmentation is performed using **thresholding** and **contour detection**.

Thresholding converts a grayscale image into a binary image. Pixels above a threshold value are assigned one value, usually white, and pixels below the threshold are assigned another value, usually black. OpenCV provides cv2.threshold() for this operation.

Contours are curves joining continuous points along the boundary of an object having the same color or intensity. They are useful for shape analysis, object detection, and object recognition.

Methodology

1. Import required Python libraries.
2. Load an image using OpenCV.
3. Convert the image from BGR to RGB for display.
4. Convert the image into grayscale.
5. Apply Gaussian blur to reduce noise.
6. Apply thresholding to create a binary image.
7. Find contours from the binary image.
8. Select the largest contour as the main object.
9. Create a mask for the region of interest.
10. Extract the segmented object using the mask.
11. Display the original image, binary image, mask, and segmented output.

Dataset

No fixed dataset is required.

A single image can be used as input.

Example image types:

- Fruit image
- Coin image
- Object on plain background
- Leaf image
- Medical image
- Vehicle image

For best results, use an image where the object and background have clear contrast.

Example file name:

object.jpg

Code

```
# Experiment: Simple Image Segmentation using Python and OpenCV

# Install OpenCV if required
# !pip install opencv-python

import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Load the image
# Replace 'object.jpg' with your image file path
image = cv2.imread("object.jpg")

# Check if image is loaded
if image is None:
    raise FileNotFoundError("Image not found. Please check the image path.")

# 2. Convert BGR image to RGB for display
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# 3. Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# 4. Apply Gaussian blur to reduce noise
blurred = cv2.GaussianBlur(gray, (5, 5), 0)

# 5. Apply thresholding
# Otsu thresholding automatically selects threshold value
_, threshold = cv2.threshold(
    blurred,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

# 6. Find contours
contours, hierarchy = cv2.findContours(
    threshold,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

# 7. Create an empty mask
mask = np.zeros_like(gray)
```

```
# 8. Find the largest contour
largest_contour = max(contours, key=cv2.contourArea)

# 9. Draw the largest contour on the mask
cv2.drawContours(
    mask,
    [largest_contour],
    -1,
    255,
    thickness=cv2.FILLED
)

# 10. Extract segmented object using mask
segmented_object = cv2.bitwise_and(image_rgb, image_rgb, mask=mask)

# 11. Draw contour boundary on original image
contour_output = image_rgb.copy()
cv2.drawContours(
    contour_output,
    [largest_contour],
    -1,
    (255, 0, 0),
    3
)

# 12. Display results
plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(threshold, cmap="gray")
plt.title("Thresholded Image")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(mask, cmap="gray")
plt.title("Object Mask")
plt.axis("off")

plt.subplot(2, 3, 5)
```

```
plt.imshow(segmented_object)
plt.title("Segmented Object")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(contour_output)
plt.title("Detected Contour")
plt.axis("off")

plt.tight_layout()
plt.show()

# 13. Print contour area
print("Number of contours detected:", len(contours))
print("Largest contour area:", cv2.contourArea(largest_contour))
```

Results

Expected output:

- Original image is displayed.
- Grayscale image is generated.
- Thresholded binary image separates foreground and background.
- Object mask is created.
- Region of interest is extracted.
- Contour boundary is drawn around the detected object.

Sample result table:

Step	**Output**
Input	Original object image
Preprocessing	Grayscale and blurred image
Segmentation	Binary thresholded image
Object Detection	Largest contour selected
Final Output	Segmented object separated from background

Outcome of the Experiment

- Image segmentation was successfully implemented using Python and OpenCV.
- The image was converted into grayscale and processed using thresholding.
- The object region was separated from the background.
- Contour detection was used to identify the object boundary.

- A mask was created to extract the region of interest.
- The experiment demonstrated how simple image segmentation can be used for object extraction and computer vision preprocessing.

Relevant Demonstrative Video Link

- Image Processing with OpenCV and Python:
- Object Detection using Image Segmentation in OpenCV:
- OpenCV Thresholding Tutorial:

Other Resource Materials / Web Links

- OpenCV Image Thresholding Tutorial: https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_thresholding/py_thresholding.html
- OpenCV Contours Tutorial: https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_contours/py_contours_begin/py_contours_begin.html
- LearnOpenCV Contour Detection using OpenCV: https://learnopencv.com/contour-detection-using-opencv-python-c/
- PyImageSearch OpenCV Thresholding: https://pyimagesearch.com/2021/04/28/opencv-thresholding-cv2-threshold/
- GeeksforGeeks OpenCV Python Tutorial: https://www.geeksforgeeks.org/python/opencv-python-tutorial/

Questions to be asked for Viva Voce:

Q1. Define image segmentation and mention its purpose in computer vision.
Q2. Explain the need of separating an object or region of interest from an image.
Q3. Describe threshold-based image segmentation.
Q4. Explain edge-based segmentation using OpenCV.
Q5. Discuss the role of grayscale conversion in image segmentation.
Q6. Explain the importance of image preprocessing before segmentation.
Q7. Identify commonly used OpenCV functions for segmentation.
Q8. Compare binary segmentation and multi-class segmentation.

Practical 12

Mapped Course Outcomes - CO5

CO5 (Create): Design and implement optimized machine learning solutions by integrating advanced supervised algorithms, unsupervised learning methods, ensemble learning, federated learning, image generation, and explainable AI for complex predictive and decision-making problems.

Manual of the experiment.

Aim

To implement a basic **Generative Adversarial Network (GAN)** to generate synthetic images from random noise.

Objective

- To understand the basic concept of Generative Adversarial Networks.
- To implement a simple GAN using Python and TensorFlow/Keras.
- To train a generator model to create synthetic images.
- To train a discriminator model to distinguish real and fake images.
- To generate handwritten digit-like images from random noise.
- To visualize generated images after training.

Tools Required

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Matplotlib
- TensorFlow / Keras

Theory

A **Generative Adversarial Network (GAN)** is a deep learning model used to generate new synthetic data that looks similar to real data. A GAN has two neural networks:

Component	Role
Generator	Generates fake images from random noise
Discriminator	Classifies images as real or fake

The generator and discriminator are trained together in an adversarial way. The generator tries to fool the discriminator by producing realistic images, while the discriminator tries to correctly identify whether an image is real or generated.

The TensorFlow DCGAN tutorial demonstrates how GANs can generate handwritten digit images using the MNIST dataset and a generator-discriminator training loop.

GAN Working Process

1. Random noise is given to the generator.
2. The generator creates a fake image.
3. Real images and fake images are given to the discriminator.
4. The discriminator learns to classify real and fake images.
5. The generator improves by learning how to fool the discriminator.
6. After training, the generator can create synthetic images.

Methodology

1. Import required Python libraries.
2. Load the MNIST handwritten digit dataset.
3. Normalize image pixel values between -1 and 1.
4. Build the generator neural network.
5. Build the discriminator neural network.
6. Define loss functions and optimizers.
7. Train discriminator using real and fake images.
8. Train generator using random noise.
9. Generate synthetic images at different epochs.
10. Visualize generated images.

Dataset

Dataset used: **MNIST Handwritten Digits Dataset**

MNIST is a popular image dataset containing handwritten digit images from 0 to 9. Each image is grayscale and has size **28 × 28 pixels**.

Property	**Value**
Dataset	MNIST
Image Size	28 × 28
Image Type	Grayscale
Classes	10 digits

Training Images	60,000
Testing Images	10,000

This experiment uses MNIST images as real images, and the GAN learns to generate similar handwritten digit-like images.

Code

```
# Experiment: Basic GAN to Generate Synthetic Images from Random Noise

import tensorflow as tf
from tensorflow.keras import layers
import numpy as np
import matplotlib.pyplot as plt

# 1. Load MNIST dataset
(X_train, _), (_, _) = tf.keras.datasets.mnist.load_data()

# 2. Normalize images to range [-1, 1]
X_train = X_train.astype("float32")
X_train = (X_train - 127.5) / 127.5

# 3. Add channel dimension
X_train = np.expand_dims(X_train, axis=-1)

print("Training data shape:", X_train.shape)

# 4. Define constants
BUFFER_SIZE = 60000
BATCH_SIZE = 256
NOISE_DIM = 100
EPOCHS = 50
NUM_EXAMPLES_TO_GENERATE = 16

# 5. Create TensorFlow dataset
train_dataset = tf.data.Dataset.from_tensor_slices(X_train)
train_dataset = train_dataset.shuffle(BUFFER_SIZE).batch(BATCH_SIZE)

# 6. Build Generator model
def build_generator():
    model = tf.keras.Sequential()

    model.add(layers.Dense(7 * 7 * 256, use_bias=False, input_shape=(NOISE_DIM,)))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Reshape((7, 7, 256)))
```

```
    model.add(layers.Conv2DTranspose(
        128, kernel_size=5, strides=1, padding="same", use_bias=False
    ))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(
        64, kernel_size=5, strides=2, padding="same", use_bias=False
    ))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(
        1, kernel_size=5, strides=2, padding="same",
        use_bias=False, activation="tanh"
    ))

    return model

# 7. Build Discriminator model
def build_discriminator():
    model = tf.keras.Sequential()

    model.add(layers.Conv2D(
        64, kernel_size=5, strides=2, padding="same",
        input_shape=[28, 28, 1]
    ))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Conv2D(
        128, kernel_size=5, strides=2, padding="same"
    ))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Flatten())
    model.add(layers.Dense(1))

    return model

# 8. Create models
generator = build_generator()
discriminator = build_discriminator()

# 9. Define loss function
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)

# Discriminator loss
```

```
def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    total_loss = real_loss + fake_loss
    return total_loss

# Generator loss
def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

# 10. Define optimizers
generator_optimizer = tf.keras.optimizers.Adam(learning_rate=1e-4)
discriminator_optimizer = tf.keras.optimizers.Adam(learning_rate=1e-4)

# 11. Function to generate and display images
def generate_and_show_images(model, epoch, test_input):
    predictions = model(test_input, training=False)

    plt.figure(figsize=(4, 4))

    for i in range(predictions.shape[0]):
        plt.subplot(4, 4, i + 1)
        image = predictions[i, :, :, 0]
        image = (image + 1) / 2.0
        plt.imshow(image, cmap="gray")
        plt.axis("off")

    plt.suptitle("Generated Images at Epoch " + str(epoch))
    plt.show()

# 12. Training step
@tf.function
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, NOISE_DIM])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator(noise, training=True)

        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)

        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)

    gradients_of_generator = gen_tape.gradient(
        gen_loss, generator.trainable_variables
    )
    gradients_of_discriminator = disc_tape.gradient(
        disc_loss, discriminator.trainable_variables
```

```
    )

    generator_optimizer.apply_gradients(
        zip(gradients_of_generator, generator.trainable_variables)
    )
    discriminator_optimizer.apply_gradients(
        zip(gradients_of_discriminator, discriminator.trainable_variables)
    )

    return gen_loss, disc_loss

# 13. Training loop
seed = tf.random.normal([NUM_EXAMPLES_TO_GENERATE, NOISE_DIM])

for epoch in range(1, EPOCHS + 1):
    for image_batch in train_dataset:
        gen_loss, disc_loss = train_step(image_batch)

    print(
        "Epoch:",
        epoch,
        "Generator Loss:",
        float(gen_loss),
        "Discriminator Loss:",
        float(disc_loss)
    )

    # Display generated images every 10 epochs
    if epoch % 10 == 0:
        generate_and_show_images(generator, epoch, seed)

# 14. Final generated images
generate_and_show_images(generator, "Final", seed)
```

Results

Sample expected observations:

Stage	**Observation**
Initial Epochs	Generated images look like random noise
Middle Epochs	Some digit-like shapes start appearing
Final Epochs	Generated images become clearer and more similar to handwritten digits
Generator Loss	Changes as generator learns to fool discriminator

Discriminator Loss	Changes as discriminator learns real/fake classification

The output images may not be perfect because this is a basic GAN. More epochs, better architecture, and hyperparameter tuning can improve image quality.

Outcome of the Experiment

- A basic GAN was successfully implemented using Python and TensorFlow/Keras.
- The generator learned to create synthetic handwritten digit-like images from random noise.
- The discriminator learned to distinguish real MNIST images from fake generated images.
- The adversarial training process was observed through generator and discriminator losses.
- Generated images were visualized at different training stages.
- The experiment demonstrated the basic working of image generation using GANs.

Relevant Demonstrative Video Link

- TensorFlow DCGAN Tutorial Video:
- GANs Explained:
- GAN Implementation using Keras:

Other Resource Materials / Web Links

- TensorFlow Official DCGAN Tutorial: https://www.tensorflow.org/tutorials/generative/dcgan
- PyTorch Official DCGAN Tutorial: https://docs.pytorch.org/tutorials/beginner/dcgan_faces_tutorial.html
- Keras Conditional GAN Example: https://keras.io/examples/generative/conditional_gan/
- Machine Learning Mastery GAN for MNIST Tutorial: https://www.machinelearningmastery.com/how-to-develop-a-generative-adversarial-network-for-an-mnist-handwritten-digits-from-scratch-in-keras/
- TensorFlow MNIST Dataset: https://www.tensorflow.org/datasets/catalog/mnist

Questions to be asked for Viva Voce:

Q1. Define Generative Adversarial Network and mention its purpose.

Q2. Explain the role of Generator in GAN architecture.

Q3. Explain the role of Discriminator in GAN architecture.

Q4. Describe adversarial training in GAN.

Q5. Explain the use of random noise in synthetic image generation.

Q6. Discuss the loss function used in GAN training.

Q7. Analyse the training process of Generator and Discriminator.

Q8. Identify challenges such as mode collapse and training instability.

Q9. Evaluate the quality of GAN-generated synthetic images.

Q10. Discuss real-world applications of GANs in image generation and data augmentation.

Practical 13

Mapped Course Outcomes - CO5

CO5 (Create): Design and implement optimized machine learning solutions by integrating advanced supervised algorithms, unsupervised learning methods, ensemble learning, federated learning, image generation, and explainable AI for complex predictive and decision-making problems.

Manual of the experiment.

Aim

To implement **SHAP** and **LIME** techniques to explain the predictions of a **Random Forest Model** on a classification task.

Objective

- To understand Explainable AI techniques.
- To train a Random Forest classifier on a classification dataset.
- To evaluate the model using accuracy, confusion matrix, and classification report.
- To apply SHAP for global and local model explanation.
- To apply LIME for explaining individual predictions.
- To compare SHAP and LIME explanations.

Tools Required

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Scikit-learn
- SHAP
- LIME

Theory

Explainable AI (XAI) is used to understand how machine learning models make predictions. Many models, such as Random Forest, XGBoost, and Neural Networks, are powerful but difficult to interpret directly.

Random Forest is an ensemble learning algorithm that builds multiple decision trees on different subsets of data and combines their outputs to improve prediction accuracy and reduce overfitting.

SHAP, or **SHapley Additive exPlanations**, explains model predictions using concepts from game theory. For tree-based models such as Random Forest, SHAP provides TreeExplainer, which uses Tree SHAP algorithms to explain ensemble tree models efficiently.

LIME, or **Local Interpretable Model-agnostic Explanations**, explains individual predictions by creating a simple interpretable model around one specific prediction. LIME can explain any black-box classifier if the classifier provides probability outputs, and it supports scikit-learn classifiers.

Methodology

1. Import required Python libraries.
2. Load a classification dataset.
3. Split the dataset into training and testing data.
4. Train a Random Forest classifier.
5. Evaluate model performance.
6. Apply SHAP TreeExplainer.
7. Generate SHAP summary plot for global explanation.
8. Generate SHAP force/waterfall plot for local explanation.
9. Apply LIME Tabular Explainer.
10. Explain one selected prediction using LIME.
11. Compare SHAP and LIME explanations.

Dataset

Dataset used: **Breast Cancer Wisconsin Dataset**

This dataset is available in Scikit-learn and is used for binary classification.

Property	**Description**
Dataset Name	Breast Cancer Wisconsin Dataset
Number of Samples	569
Number of Features	30
Number of Classes	2
Classes	Malignant, Benign
Task	Tumor classification

The goal is to classify whether a tumor is malignant or benign using numerical medical features.

Code

```
# Experiment: SHAP and LIME for Random Forest Classification

# Install required libraries if needed
# !pip install shap lime

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

import shap
from lime.lime_tabular import LimeTabularExplainer

# 1. Load dataset
data = load_breast_cancer()

X = data.data
y = data.target

feature_names = data.feature_names
class_names = data.target_names

# 2. Create DataFrame
df = pd.DataFrame(X, columns=feature_names)
df["target"] = y

print("First five rows of dataset:")
print(df.head())

print("\nDataset shape:", df.shape)
print("Classes:", class_names)

# 3. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
```

```
)

# 4. Train Random Forest model
rf_model = RandomForestClassifier(
    n_estimators=100,
    max_depth=5,
    random_state=42
)

rf_model.fit(X_train, y_train)

# 5. Predict test data
y_pred = rf_model.predict(X_test)

# 6. Evaluate model
accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy:", accuracy)

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=class_names))

# ------------------------------------------------------
# SHAP Explanation
# ------------------------------------------------------

# 7. Create SHAP explainer
explainer = shap.TreeExplainer(rf_model)

# 8. Calculate SHAP values
shap_values = explainer.shap_values(X_test)

# For binary classification, select class 1 explanation
# Depending on SHAP version, shap_values may be list or array
if isinstance(shap_values, list):
    shap_values_class1 = shap_values[1]
else:
    shap_values_class1 = shap_values[:, :, 1]

# 9. SHAP summary plot
shap.summary_plot(
    shap_values_class1,
    X_test,
    feature_names=feature_names
)
```

```
# 10. SHAP bar plot for feature importance
shap.summary_plot(
    shap_values_class1,
    X_test,
    feature_names=feature_names,
    plot_type="bar"
)

# 11. SHAP local explanation for one instance
sample_index = 0

print("\nActual Class:", class_names[y_test[sample_index]])
print("Predicted Class:", class_names[y_pred[sample_index]])

# Waterfall plot for selected sample
# This works in newer SHAP versions
try:
    explanation = explainer(X_test)

    if len(explanation.values.shape) == 3:
        single_explanation = shap.Explanation(
            values=explanation.values[sample_index, :, 1],
            base_values=explanation.base_values[sample_index, 1],
            data=X_test[sample_index],
            feature_names=feature_names
        )
    else:
        single_explanation = explanation[sample_index]

    shap.plots.waterfall(single_explanation)

except Exception as e:
    print("Waterfall plot could not be displayed:", e)

# ----------------------------------------------------------
# LIME Explanation
# ----------------------------------------------------------

# 12. Create LIME explainer
lime_explainer = LimeTabularExplainer(
    training_data=X_train,
    feature_names=feature_names,
    class_names=class_names,
    mode="classification",
    discretize_continuous=True,
    random_state=42
)

# 13. Explain one prediction
```

```
lime_exp = lime_explainer.explain_instance(
    data_row=X_test[sample_index],
    predict_fn=rf_model.predict_proba,
    num_features=10
)

# 14. Display LIME explanation in notebook
lime_exp.show_in_notebook(show_table=True)

# 15. Print LIME explanation as list
print("\nLIME Explanation:")
for feature, weight in lime_exp.as_list():
    print(feature, ":", weight)

# 16. Plot LIME explanation
fig = lime_exp.as_pyplot_figure()
plt.title("LIME Explanation for One Prediction")
plt.show()
```

Results

Sample expected model result:

Metric	**Value**
Accuracy	Around 0.94 to 0.97
Model	Random Forest Classifier
Dataset	Breast Cancer Wisconsin

Sample confusion matrix:

	Predicted Malignant	**Predicted Benign**
Actual Malignant	40	2
Actual Benign	3	69

SHAP Result

- SHAP summary plot shows the most important features affecting model predictions.
- SHAP bar plot ranks features based on average importance.
- SHAP local explanation shows how each feature increases or decreases the prediction for one sample.

Common important features may include:

- Worst radius
- Worst perimeter
- Worst area

- Mean concave points
- Worst concave points

LIME Result

- LIME explains one selected prediction.
- It shows which feature conditions supported or opposed the predicted class.
- Positive feature weights support the predicted class.
- Negative feature weights oppose the predicted class.

Outcome of the Experiment

- A Random Forest classifier was successfully implemented.
- The classification model was evaluated using accuracy, confusion matrix, and classification report.
- SHAP was used to explain both global and local model behavior.
- LIME was used to explain an individual prediction.
- SHAP helped identify overall important features in the model.
- LIME helped understand why a particular sample was classified into a specific class.
- The experiment demonstrated the importance of Explainable AI in making machine learning models more transparent and trustworthy.

Relevant Demonstrative Video Link

- SHAP Values Explained:
- LIME Model Explainability in Python:
- Explainable AI: LIME and SHAP:

Other Resource Materials / Web Links

- SHAP TreeExplainer Documentation: https://shap.readthedocs.io/en/latest/generated/shap.TreeExplainer.html
- LIME GitHub Repository: https://github.com/marcotcr/lime
- LIME Tabular Explainer Documentation: https://lime-ml.readthedocs.io/en/latest/lime.html

- Scikit-learn RandomForestClassifier Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html
- Scikit-learn Breast Cancer Dataset Documentation: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html

Questions to be asked for Viva Voce:

Q1. Define Explainable AI and mention its importance in machine learning.

Q2. Explain the need of SHAP and LIME for black-box model interpretation.

Q3. Describe the working of LIME for explaining individual predictions.

Q4. Explain the working of SHAP using feature contribution.

Q5. Differentiate SHAP and LIME.

Q6. Discuss the role of Random Forest as a black-box model.

Q7. Identify important features using SHAP values.

Q8. Analyze local explanations generated by LIME.

Q9. Compare global and local explainability in model interpretation.

Q10. Evaluate the importance of SHAP and LIME in trustworthy AI systems.

Practical 14

Mapped Course Outcomes - CO5

CO5 (Create): Design and implement optimized machine learning solutions by integrating advanced supervised algorithms, unsupervised learning methods, ensemble learning, federated learning, image generation, and explainable AI for complex predictive and decision-making problems.

Manual of the experiment.

Aim

To apply an **object detection model** to detect and classify objects in an image dataset.

Objective

- To understand the concept of object detection.
- To use a pre-trained object detection model.
- To detect objects, present in an image.
- To classify detected objects using class labels.
- To draw bounding boxes around detected objects.
- To display confidence scores for each detected object.
- To evaluate the usefulness of object detection in real-world applications.

Tools Required

- Python
- Jupyter Notebook / Google Colab
- OpenCV
- Matplotlib
- Ultralytics YOLO
- NumPy

Theory

Object detection is a computer vision task that identifies and locates objects in an image or video. Unlike image classification, which only predicts the class of the whole image, object detection predicts both:

- **Class label** of the object
- **Bounding box** location of the object

For example, in a street image, an object detection model can detect and classify objects such as person, car, bus, bicycle, traffic light, and dog.

In this experiment, a **YOLO model** is used. YOLO stands for **You Only Look Once**. YOLO detects objects in a single pass through the image, making it fast and suitable for real-time applications. Ultralytics YOLO supports prediction on images, videos, and live streams using pre-trained models.

The model used here is trained on the **COCO dataset**, which is a large-scale dataset for object detection, segmentation, and captioning. COCO contains images with annotations for common object categories.

Methodology

1. Import required Python libraries.
2. Install and load the YOLO object detection model.
3. Load an input image.
4. Pass the image to the YOLO model.
5. Detect objects in the image.
6. Extract class names, confidence scores, and bounding boxes.
7. Draw bounding boxes around detected objects.
8. Display detected object names and confidence scores.
9. Save or display the final output image.
10. Analyze the detection result.

Dataset

Dataset used: **COCO Dataset / Custom Image Dataset**

For this experiment, a pre-trained YOLO model trained on the COCO object detection dataset is used.

COCO supports common object classes such as:

- Person
- Bicycle
- Car
- Motorcycle
- Bus
- Truck
- Dog

- Cat
- Chair
- Bottle
- Laptop
- Cell phone

The input image can be any image containing objects from COCO classes.

Example image file:

image.jpg

You can use:

- A single image
- A folder of images
- COCO sample images
- Your own custom image dataset

Code

```
# Experiment: Object Detection using YOLO and Python

# Install required library
# Run this command in Google Colab or Jupyter Notebook if not installed:
# !pip install ultralytics opencv-python matplotlib

from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt
import os

# 1. Load pre-trained YOLO model
# yolov8n.pt is a lightweight pre-trained YOLO model
model = YOLO("yolov8n.pt")

# 2. Provide input image path
image_path = "image.jpg"

# Check image path
if not os.path.exists(image_path):
    raise FileNotFoundError("Image not found. Please check the image path.")

# 3. Read image using OpenCV
image = cv2.imread(image_path)
```

```
# Convert BGR image to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# 4. Run object detection
results = model(image_rgb)

# 5. Get detected result
result = results[0]

# 6. Draw bounding boxes on image
annotated_image = result.plot()

# Convert annotated image from BGR to RGB if required
annotated_image = cv2.cvtColor(annotated_image, cv2.COLOR_BGR2RGB)

# 7. Display original and detected image
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(annotated_image)
plt.title("Detected Objects")
plt.axis("off")

plt.show()

# 8. Print detected object details
print("Detected Objects:")
print("-----------------")

for box in result.boxes:
    class_id = int(box.cls[0])
    confidence = float(box.conf[0])
    class_name = model.names[class_id]

    x1, y1, x2, y2 = box.xyxy[0]

    print("Class:", class_name)
    print("Confidence:", round(confidence, 2))
    print("Bounding Box:", [round(float(x1), 2), round(float(y1), 2),
                            round(float(x2), 2), round(float(y2), 2)])
    print("-----------------")

# 9. Save output image
output_path = "detected_output.jpg"
```

```
cv2.imwrite(output_path, cv2.cvtColor(annotated_image, cv2.COLOR_RGB2BGR))

print("Output image saved as:", output_path)
```

Code for Detecting Objects in a Folder of Images

```
from ultralytics import YOLO
import os

# Load YOLO model
model = YOLO("yolov8n.pt")

# Folder containing images
input_folder = "images"

# Output folder
output_folder = "detected_results"
os.makedirs(output_folder, exist_ok=True)

# Run detection on all images in folder
results = model.predict(
    source=input_folder,
    save=True,
    conf=0.25
)

print("Object detection completed.")
print("Detected images are saved automatically in the runs/detect folder.")
```

Results

Sample expected output:

Object Detected	Confidence Score	Bounding Box
Person	0.91	x1, y1, x2, y2
Car	0.87	x1, y1, x2, y2
Bus	0.84	x1, y1, x2, y2
Dog	0.76	x1, y1, x2, y2

Expected visual result:

- Objects are detected in the image.
- Each object is surrounded by a bounding box.
- Each bounding box displays class name and confidence score.
- The final detected image is saved as detected_output.jpg.

Outcome of the Experiment

- A pre-trained YOLO object detection model was successfully applied.
- Objects present in an input image were detected and classified.
- Bounding boxes were drawn around detected objects.
- Confidence scores were displayed for each prediction.
- The experiment demonstrated the use of object detection for real-world computer vision tasks.
- The model can be extended to video detection, webcam detection, and custom object detection.

Relevant Demonstrative Video Link

- YOLO Object Detection using Python:
  YOLOv8 Object Detection Tutorial:
  Object Detection with OpenCV and Python:

Other Resource Materials / Web Links

- Ultralytics YOLO Documentation:
  https://docs.ultralytics.com/
- Ultralytics YOLO Predict Mode:
  https://docs.ultralytics.com/modes/predict/
- Ultralytics YOLO Python Usage:
  https://docs.ultralytics.com/usage/python/
- COCO Dataset Official Website:
  https://cocodataset.org/
- OpenCV Python Package:
  https://pypi.org/project/opencv-python/
- OpenCV GitHub Repository:
  https://github.com/opencv/opencv/

Questions to be asked for Viva Voce:

Q1. Define object detection in computer vision.

Q2. Differentiate image classification, object localization and object detection.

Q3. Explain the role of bounding boxes in object detection.

Q4. Describe the role of class labels and confidence score.

Q5. Explain the working of an object detection model.

Q6. Discuss the use of YOLO for object detection.

Q7. Compare single-stage and two-stage object detection models.

Q8. Identify evaluation metrics used in object detection.

Practical 15

Mapped Course Outcomes - CO5

CO5 (Create): Design and implement optimized machine learning solutions by integrating advanced supervised algorithms, unsupervised learning methods, ensemble learning, federated learning, image generation, and explainable AI for complex predictive and decision-making problems.

Manual of the experiment.

Aim

To implement **vehicle detection in road traffic** using the concept of **object detection**.

Objective

- To understand object detection in traffic monitoring.
- To apply a pre-trained YOLO object detection model.
- To detect vehicles such as cars, buses, trucks, motorcycles, and bicycles.
- To draw bounding boxes around detected vehicles.
- To classify detected vehicles using object labels.
- To display confidence scores for each detected vehicle.
- To understand the use of vehicle detection in intelligent transportation systems.

Tools Required

- Python
- Jupyter Notebook / Google Colab
- OpenCV
- Matplotlib
- NumPy
- Ultralytics YOLO

Theory

Vehicle detection is a computer vision application used to identify and locate vehicles in road traffic images or videos. It is commonly used in smart traffic management, vehicle counting, road safety monitoring, traffic rule enforcement, and surveillance systems.

Object detection performs two main tasks:

1. **Classification**: Identifies the object category, such as car, bus, truck, motorcycle, or bicycle.
2. **Localization**: Finds the position of the object using a bounding box.

In this experiment, a **YOLO** object detection model is used. YOLO stands for **You Only Look Once**. It detects objects in a single forward pass, making it suitable for real-time detection tasks. Ultralytics YOLO supports prediction on images, videos, folders, and live streams.

The pre-trained YOLO model is trained on the **COCO dataset**, which contains 80 object categories, including common traffic-related objects such as cars, bicycles, motorcycles, buses, and trucks.

Methodology

1. Import required Python libraries.
2. Install and load the YOLO object detection model.
3. Load a road traffic image.
4. Pass the image to the YOLO model.
5. Detect all objects present in the image.
6. Filter only vehicle classes:
    - Car
    - Bus
    - Truck
    - Motorcycle
    - Bicycle
7. Draw bounding boxes around detected vehicles.
8. Display class name and confidence score.
9. Count total detected vehicles.
10. Save and display the final output image.

Dataset

A road traffic image dataset or a single road traffic image can be used.

Example input:

traffic.jpg

The image may contain:

- Cars
- Buses

- Trucks
- Motorcycles
- Bicycles
- Pedestrians
- Traffic lights

For this experiment, a pre-trained YOLO model trained on the COCO dataset is used. COCO is a large-scale dataset for object detection, segmentation, and captioning.

Code

```
# Experiment: Vehicle Detection in Road Traffic using YOLO

# Install required libraries if not already installed
# !pip install ultralytics opencv-python matplotlib

from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt
import os

# 1. Load pre-trained YOLO model
model = YOLO("yolov8n.pt")

# 2. Input traffic image path
image_path = "traffic.jpg"

# 3. Check if image exists
if not os.path.exists(image_path):
    raise FileNotFoundError("Image not found. Please check the image path.")

# 4. Read image using OpenCV
image = cv2.imread(image_path)

# 5. Convert BGR image to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# 6. Run object detection
results = model(image_rgb)

# 7. Get first result
result = results[0]

# 8. Vehicle classes in COCO dataset
vehicle_classes = ["car", "bus", "truck", "motorcycle", "bicycle"]
```

```
# 9. Copy image for drawing output
output_image = image_rgb.copy()

# 10. Initialize vehicle count
vehicle_count = 0

print("Detected Vehicles:")
print("------------------")

# 11. Process detected boxes
for box in result.boxes:
    class_id = int(box.cls[0])
    confidence = float(box.conf[0])
    class_name = model.names[class_id]

    # Filter only vehicle classes
    if class_name in vehicle_classes:
        vehicle_count += 1

        # Get bounding box coordinates
        x1, y1, x2, y2 = box.xyxy[0]
        x1, y1, x2, y2 = int(x1), int(y1), int(x2), int(y2)

        # Draw bounding box
        cv2.rectangle(
            output_image,
            (x1, y1),
            (x2, y2),
            (255, 0, 0),
            2
        )

        # Label with class name and confidence
        label = f"{class_name}: {confidence:.2f}"

        cv2.putText(
            output_image,
            label,
            (x1, y1 - 10),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.6,
            (255, 0, 0),
            2
        )

        print("Class:", class_name)
        print("Confidence:", round(confidence, 2))
        print("Bounding Box:", [x1, y1, x2, y2])
        print("------------------")
```

```
# 12. Display total vehicle count
print("Total Vehicles Detected:", vehicle_count)

# 13. Display original and output images
plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Traffic Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(output_image)
plt.title("Detected Vehicles")
plt.axis("off")

plt.show()

# 14. Save output image
output_path = "vehicle_detection_output.jpg"
cv2.imwrite(output_path, cv2.cvtColor(output_image, cv2.COLOR_RGB2BGR))

print("Output image saved as:", output_path)
```

Code for Vehicle Detection in a Traffic Video

```
# Vehicle Detection in Road Traffic Video using YOLO

from ultralytics import YOLO
import cv2

# Load YOLO model
model = YOLO("yolov8n.pt")

# Input video path
video_path = "traffic_video.mp4"

# Open video
cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    raise FileNotFoundError("Video not found. Please check the video path.")

# Vehicle classes
vehicle_classes = ["car", "bus", "truck", "motorcycle", "bicycle"]

while True:
```

```
ret, frame = cap.read()

if not ret:
    break

# Run detection
results = model(frame)

result = results[0]

vehicle_count = 0

for box in result.boxes:
    class_id = int(box.cls[0])
    confidence = float(box.conf[0])
    class_name = model.names[class_id]

    if class_name in vehicle_classes:
        vehicle_count += 1

        x1, y1, x2, y2 = box.xyxy[0]
        x1, y1, x2, y2 = int(x1), int(y1), int(x2), int(y2)

        cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)

        label = f"{class_name}: {confidence:.2f}"
        cv2.putText(
            frame,
            label,
            (x1, y1 - 10),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.6,
            (0, 255, 0),
            2
        )

cv2.putText(
    frame,
    f"Vehicle Count: {vehicle_count}",
    (30, 40),
    cv2.FONT_HERSHEY_SIMPLEX,
    1,
    (0, 0, 255),
    2
)

cv2.imshow("Vehicle Detection", frame)

if cv2.waitKey(1) & 0xFF == ord("q"):
```

```
    break

cap.release()
cv2.destroyAllWindows()
```

Results

Expected output:

- Road traffic image is loaded successfully.
- Vehicles are detected and classified.
- Bounding boxes are drawn around vehicles.
- Class labels and confidence scores are displayed.
- Total number of vehicles is printed.

Sample result table:

Detected Object	**Confidence Score**	**Output**
Car	0.91	Bounding box drawn
Bus	0.87	Bounding box drawn
Truck	0.82	Bounding box drawn
Motorcycle	0.78	Bounding box drawn

Example output:

```
Detected Vehicles:
Class: car
Confidence: 0.91
Bounding Box: [120, 80, 250, 190]

Class: bus
Confidence: 0.87
Bounding Box: [300, 100, 520, 260]

Total Vehicles Detected: 2
```

Outcome of the Experiment

- Vehicle detection was successfully implemented using YOLO.
- Different vehicle types such as cars, buses, trucks, motorcycles, and bicycles were detected.
- Bounding boxes were drawn around detected vehicles.
- Confidence scores were displayed for each detected vehicle.

- The total number of vehicles was calculated.
- The experiment demonstrated the use of object detection in road traffic monitoring and intelligent transportation systems.

Relevant Demonstrative Video Link

- YOLOv8 Vehicle Detection and Tracking:
- Vehicle Counting using YOLOv8 and SORT:
- YOLO Object Detection using Python:

Other Resource Materials / Web Links

- Ultralytics YOLO Predict Documentation: https://docs.ultralytics.com/modes/predict/
- Ultralytics YOLO Python Usage: https://docs.ultralytics.com/usage/python/
- Ultralytics COCO Dataset Documentation: https://docs.ultralytics.com/datasets/detect/coco/
- COCO Dataset Official Website: https://cocodataset.org/
- OpenCV Python Documentation: https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html
- Vehicle Dataset for YOLO: https://datasetninja.com/vehicle-dataset-for-yolo

Questions to be asked for Viva Voce:

Q1. Define vehicle detection in road traffic monitoring.

Q2. Explain the importance of vehicle detection in intelligent transportation systems.

Q3. Describe the use of object detection for identifying vehicles in traffic images.

Q4. Explain the role of bounding boxes in vehicle detection.

Q5. Discuss the challenges of vehicle detection in crowded traffic scenes.

Q6. Analyze the effect of lighting, weather and occlusion on vehicle detection accuracy.

Q7. Compare vehicle detection in images and videos.

Q8. Identify suitable object detection models for road traffic analysis.

Q9. Evaluate the role of vehicle detection in traffic management and accident prevention.

Q10. Discuss the future scope of vehicle detection using deep learning and computer vision.